#show table.cell: set par(justify: false) #show table.cell: set text(size: 9pt, hyphenate: false)

ФСП Таланты

Платформа подбора ИТ-специалистов с подтверждённой категорией. Сопроводительная документация

Содержание

ФСП Таланты — платформа подбора ИТ-специалистов с подтверждённой категорией 2
Коротко о решении 2
1. Функциональная архитектура 3
2. Компонентная архитектура 7
3. Справочники 8
4. Механика тестирования 8
5. Механика подбора и категоризация 11
6. Процедура валидации и результаты 14
7. Интеграция с ФСП 20
8. Безопасность и 152-ФЗ 22
9. API 24
10. Антифейк и концепции вне MVP 25
11. Библиотеки и компоненты 33
12. Сборка, развёртывание и запуск 35

ФСП Таланты — платформа подбора ИТ-специалистов с подтверждённой категорией

Хакатон «Лидеры цифровой трансформации 2026», спецтрек Федерации спортивного программирования России.

Репозиторий: https://github.com/K1NGSM4N3/fsp-hack · Swagger UI развёрнутого решения: `/api/docs` · Отчёты валидации: `docs/validation/`

Коротко о решении

Работодатель не ждёт откликов, а сам выходит на кандидата. Кандидаты разложены по **категориям** — специализация × грейд, — и категорию даёт только **адаптивный тест**, а не резюме. Внутри категории выше стоят те, у кого сильнее результат теста, подтверждены навыки и есть достижения ФСП. Работодатель описывает потребность, получает **объяснённую подборку** и отправляет **приглашение с обязательной зарплатной вилкой**. Кандидат видит условия до начала общения, а его контакты открываются работодателю только после согласия.

Что отличает решение:

1. **Тест, который бесполезно сливать.** Задания — параметрические шаблоны: у каждого кандидата свои числа, код и ответы, а ответ вычисляется генератором. Сложность сопоставима, потому что шаблоны откалиброваны по IRT и все кандидаты измеряются на одной шкале. В симуляции утечки у фиксированного теста при 50 % слитых ответов проходят 99,9 % слабых кандидатов, у нашего — 9–10 %, как и без утечки (§6.2).
2. **Механика контакта «работодатель ⟶ кандидат».** Анонимная карточка, объяснение попадания, вилка в первом сообщении, квота приглашений против спама, причины отказов обратно работодателю, статистика спроса кандидату (§5).
3. **Измеримое качество.** Собственная процедура валидации на синтетике со скрытой «истиной»: доля релевантных в топ-10 — 0,94 против 0,43 у поиска по ключевым словам резюме; итоговый грейд совпадает с истинным в 87 % случаев, при пересдаче — $\kappa = 0{,}74$ (§6). Свой набор вакансий, резюме и разметки можно прогнать через рабочий подбор одной командой или из Swagger (§6.4).
4. **ФСП ID на Keycloak уже сейчас.** Привязка через OIDC + PKCE к заглушке ФСП ID на настоящем Keycloak 26, достижения из реестра влияют на ранжирование, кандидат без истории ФСП не штрафуется (§7).

5. **Видеозапись теста — по согласию и без потери анонимности.** Камера предлагается перед тестом, запись шифруется и хранится 90 дней; кандидат видит свою запись, работодатель — только отметку «Тест с видеозаписью» и число уходов со вкладки. Спорный результат разбирает модератор: подтверждает, аннулирует с восстановлением прежней категории или назначает пересдачу с камерой — каждое решение и каждый просмотр записи попадают в журнал (§4.7, §8.2).
6. **Своя ML-модель смысловой близости — на нашем сервере.** Эмбеддинги multilingual-e5-small считаются локально, тексты не уходят наружу: модель ловит пересказанные копии чужих вакансий, которых не видит сравнение по словам (половина пересказов без единого ложного срабатывания на калибровочном наборе), подбирает «Похожие вакансии» и даёт подбору объяснимый признак «Близость описаний». Прибавки к метрикам подбора от неё нет — тест и подтверждённые навыки уже несут этот сигнал, — и мы это показываем (§5.6).

1. Функциональная архитектура

1.1. Роли и сквозные сценарии

Роль	Сценарий
Кандидат	Регистрация по email с подтверждением и согласиями (с 14 лет; до 18 — с согласием законного представителя по ссылке из письма) ⟶ профиль (форма или PDF-резюме с распознаванием) ⟶ опрос (направление, специализация, стек, заявленный грейд) ⟶ адаптивный тест ⟶ категория ⟶ входящие приглашения с вилкой ⟶ принять / отклонить с причиной. Дополнительно: вакансии и отклики, регулярные задания, привязка ФСП ID, PDF-профиль, настройки приватности и согласий, журнал «Кто открывал ваши контакты», выгрузка своих данных, уведомления в кабинете и на почту
Модератор	Кабинет `/admin`: очередь тестов, где нетипичные ответы совпали с уходами со вкладки ⟶ карточка теста с ответами по сложности, лентой событий и записью ⟶ решение с комментарием (подтвердить, аннулировать, назначить пересдачу с камерой). Очередь антифейка и жалоб ⟶ снять отметку, снять вакансию, приостановить приглашения компании, вернуть. Журнал всех действий и просмотров записей
Работодатель	Профиль компании ⟶ описание потребности (свободным текстом, система распознаёт специализацию, грейд, стек и вилку) ⟶ рекомендованные категории ⟶ подборка с объяснениями, уточнение фильтров без потери ⟶ анонимный профиль ⟶ приглашение с вилкой ⟶ контакты после принятия. Дополнительно: поиск по всему банку, вакансии и отклики, регулярные задания, статистика приглашений, раздел «Доверие и квота», подключение своей ATS (вебхуки и API-ключи), уведомления об ответах и откликах

1.2. Покрытие требований ТЗ

Требование	Реализация
Категория = специализация × грейд по опросу и тесту, не по резюме	6 специализаций × 3 грейда; категорию присваивает только сданный тест (§4, §5.3)
Механика тестирования, устойчивая к утечкам, с сопоставимой сложностью	62 параметрических шаблона, IRT 3PL, адаптивный тест (§4)
Грейд не понижается принудительно; пересдача ниже / выше; ограничение по времени	Политика грейда: шаг вниз сразу после провала, дотест вверх после уверенной сдачи, смена грейда не чаще раза в 90 дней; проваленный или брошенный тест закрывает свой грейд на 30 дней (§4.5)
Ранжирование внутри категории по тесту и ФСП	Сила профиля = 0,6 · тест + 0,25 · ФСП + 0,15 · актуальность (§5.3)
Работодатель находит категорию и выходит на кандидата с предложением и зарплатой	Рекомендованные категории, подборка, поиск по банку, приглашение без привязки к вакансии (§5)
Объяснимость выдачи; уточнение без потери подборки	Разложение балла по признакам, причины и пробелы; подборка хранится, закреплённые кандидаты переживают уточнение (§5.4)
Зарплата «от–до» обязательна; контакты после принятия / отклика	Проверяется в API и ограничениях БД; контакты недоступны на уровне типов (§8.3)
Статусы приглашений и откликов	отправлено ⟶ просмотрено ⟶ принято / отклонено; истекло, отозвано
152-ФЗ: согласия на обработку и публикацию, права по ролям	Версионированные согласия; профиль публикуется только при действующем согласии; соискатели 14–17 лет — только с согласием законного представителя; политика обработки `/privacy`; выгрузка своих данных и журнал доступа к контактам в кабинете (§8.2)
Связь с ФСП ID; корректная обработка без истории ФСП	OIDC + PKCE к Keycloak, реестр достижений, навыки, подтверждённые результатами ФСП, нейтральное «Нет данных ФСП»; членство в ФСП не учитывается (§7)
PDF-резюме с распознаванием, стандартизированный PDF-профиль	Извлечение текста + словари и правила; PDF в фирменном шрифте ФСП
Регулярные задания, вакансии и отклики (желательно)	Реализованы
Защита от фиктивных вакансий (концепция, рабочая часть — плюс)	Работает: подтверждение корпоративного домена, сверка вилки с рынком категории, поиск почти одинаковых вакансий и пересказов чужих вакансий по смыслу, уровень доверия компании, от которого зависит квота приглашений, модерация с решениями и журналом: снять отметку, снять вакансию, приостановить приглашения (§10.1). Дальше — ЕГРЮЛ
ML и NLP для разбора резюме, категоризации и подбора (преимущество)	Словарь навыков с синонимами и правила разбора резюме и вакансий; IRT-модель теста; локальная модель эмбеддингов multilingual-e5-small: близость описаний в подборе, пересказы вакансий в антифейке, похожие вакансии — с честной валидацией (§5.6, §6.3)
Интеграция с ATS работодателей (концепция)	Реализован универсальный слой: подписанные вебхуки из транзакционного outbox, API-ключи со скоупами, этапы найма от ATS, импорт вакансий, демо-ATS; план нативных коннекторов (§10.2)

1.3. Интерфейс

Публичный сайт — пять страниц, у каждой свой адресат: главная (`/`) показывает механику на примере одного приглашения; `/seekers` — путь соискателя от теста до приглашения с вилкой и мини-демо адаптивного теста; `/business` — как компания описывает потребность и получает подборку, правила для всех компаний; `/fsp` — что даёт привязка ФСП ID и как разряды и призовые места попадают в профиль; `/vacancies` — опубликованные вакансии с фильтрами. Отдельно — политика обработки персональных данных `/privacy` с действующими текстами согласий и страница законного представителя `/guardian/confirm`. Страницы рендерятся на сервере и читаются без JavaScript; анимации загружаются после первой отрисовки, на телефонах облегчены, при `prefers-reduced-motion` отключены (§2).

Кабинет кандидата: главная (одна кнопка следующего шага: заполнить профиль → пройти опрос и тест → ответить на приглашения или посмотреть вакансии; категория, приглашения, спрос в категории, ФСП), профиль и резюме, опрос (грейд для теста спрашивается здесь, на странице теста он уже выбран и меняется одной кнопкой), тестирование с шагом видеозаписи, приглашения, вакансии и отклики, регулярные задания, настройки (статус поиска, видимость, ФСП ID, согласия, «Кто открывал ваши контакты», выгрузка данных, удаление аккаунта).

Кабинет работодателя: главная (квота, воронка приглашений, причины отказов), потребности и подборки, поиск по базе (на телефоне фильтры свёрнуты за кнопкой «N фильтров»), приглашения, вакансии и отклики, регулярные задания, интеграции, компания, доверие и квота (с решениями модератора о компании). Списки, которые растут, — приглашения, отклики, ответы на задания, журнал доставок — подгружаются страницами по кнопке «Показать ещё».

Кабинет модератора (роль платформы, регистрацией не выдаётся): обзор со счётчиками очередей, прокторинг — очередь и карточка теста с решениями, антифейк и жалобы (по компаниям), журнал с поиском по коду кандидата. Кабинет не индексируется.

Вёрстка адаптивная от 320 px; кабинеты на телефоне работают с выдвижным меню.

1.4. Уведомления

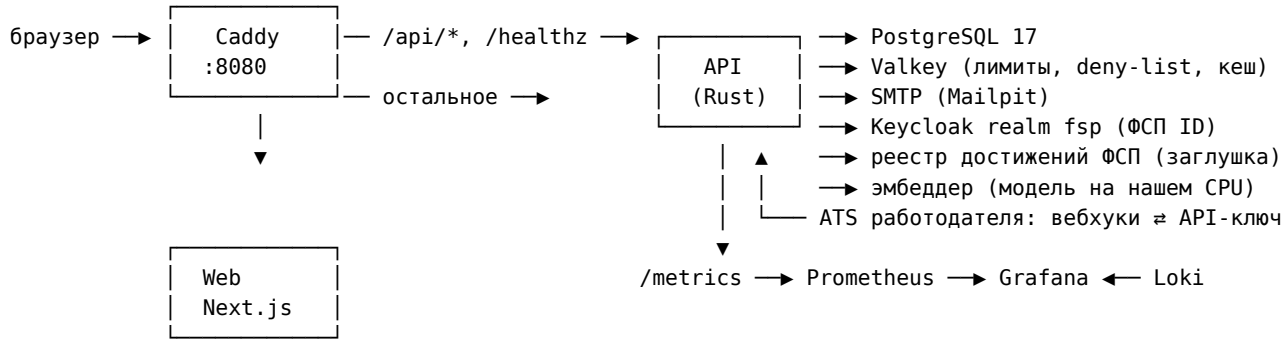
Раз компания пишет первой, соискатель должен узнать о приглашении, не заходя в кабинет, а компания — об ответе. В шапке обоих кабинетов — колокольчик со счётчиком непрочитанного и списком последних событий; полная лента — на отдельной странице, настройки писем — в настройках соискателя и на странице компании.

Кому	О чём
Соискателю	новое приглашение (компания, должность, вилка, срок ответа); напоминание за 48 часов до истечения, если не ответил, — один раз; компания отозвала приглашение; этап найма от компании или её ATS (собеседование, оффер, найм, отказ); ответ на отклик; итог теста — присвоена категория или нет; решение законного представителя (14–17 лет); решение модератора по спорному тесту — результат подтверждён, аннулирован или назначена пересдача с видеозаписью («можно пройти сразу»), всегда с комментарием модератора; новые достижения из реестра ФСП
Компании	кандидат принял приглашение — «Кандидат К-… принял приглашение — контакты в кабинете», без имени и контактов; кандидат отклонил — с причиной; новый отклик; вебхук в ATS не принимает события; изменились уровень доверия и квота; модератор снял вакансию с публикации или приостановил приглашения до даты — с комментарием; мера отменена; жалоба не подтвердилась — без автора жалобы

- **Ничего не теряется и не придумывается.** Уведомление записывается в той же транзакции, что и само событие (как события для ATS, §10.2): нет приглашения — нет и уведомления, и наоборот. Решение модератора — так же: решение, строка журнала модерации и уведомление записываются вместе или не записываются вовсе.
- **Письма без спама.** Всё, что пришло за две минуты, уходит одним письмом; получателю — не чаще письма в 10 минут и не больше 10 в сутки, лишнее ждёт и собирается в следующее письмо. Письмо — текст и простой HTML на русском, у каждого события ссылка в кабинет.
- **Приглашение — без очереди.** Новое приглашение не ждёт 10-минутной паузы: письмо уходит через две минуты сбора и забирает всё, что накопилось; суточный лимит действует и для него. Письмо начинается с самого нового приглашения — и тема тоже: «Северный Код приглашает: Rust-разработчик (и ещё 1 уведомление)».

- **Отписка в один клик.** В каждом письме — заголовки `List-Unsubscribe` и `List-Unsubscribe-Post` (почтовый клиент отписывает сам, RFC 8058) и ссылка на страницу с кнопкой «Отписаться» — без входа, по подписанной ссылке. В кабинете уведомления остаются всегда, письма включаются и выключаются по видам.
- **Счётчик дешёвый.** Кабинет спрашивает число непрочитанных раз в 30 секунд и не спрашивает из скрытой вкладки; на сервере это подсчёт по частичному индексу. О новом приглашении, пока соискатель в кабинете, — одно короткое всплывающее сообщение.

2. Компонентная архитектура



Backend — модульный монолит на Rust. Логика вынесена в чистые крейты без ввода-вывода и часов, поэтому одна и та же реализация работает в API и в симуляторе валидации:

Крейт	Назначение
`catalog`	Справочники: направления, специализации, грейды и их пороги на шкале θ, категории, отрасли
`assessment`	Тестирование: модель IRT, адаптивный выбор заданий, 62 шаблона, политика смены грейда
`matching`	Подбор: словарь навыков с синонимами и графом близости, скоринг с объяснениями, ФСП-скор, разбор текста вакансии и резюме
`db`	Доступ к PostgreSQL на sqlx: SQL-запросы, проверенные по схеме при компиляции, миграции
`api`	HTTP API (axum), аутентификация, OpenAPI, сервисы
`fsp-mock`	Заглушка реестра достижений ФСП
`ats-mock`	Демо-ATS работодателя: принимает подписанные вебхуки, отправляет этапы найма обратно (§10.2)
`embedder`	Эмбеддинги текстов: модель multilingual-e5-small на candle (CPU), HTTP-сервис `POST /embed` (§5.6)
`validate`	Процедура валидации, формирует `docs/validation/`

Frontend — Next.js 16 (App Router, TypeScript). Публичные страницы рендерятся на сервере, кабинеты — клиентские, данные через TanStack Query; типы API генерируются из OpenAPI. Права проверяет только API. Анимации публичного сайта (GSAP, Lenis) приходят отдельным чанком после гидратации и работают в двух профилях: на десктопе — пины, прокрутка со scrub, разбивка заголовков, плавный скролл; на телефонах и планшетах — одноразовые появления только через transform и opacity и нативный скролл.

Данные: PostgreSQL — пользователи (email и ФИО зашифрованы), профили, навыки с источником (заявлен / из резюме / подтверждён тестом), сессии тестов (хранятся только ссылки на экземпляры заданий: шаблон, уровень, seed), история грейдов, потребности и сохранённые подборки, приглашения и их история, журнал доступа к контактам, согласия и запросы законным представителям, метаданные видеозаписей (сами фрагменты — зашифрованными файлами на диске), очередь событий для ATS, векторы профилей, потребностей и вакансий (pgvector) с очередью на их расчёт. Схема — 17 миграций sqlx, применяются при старте API.

Развёртывание: docker compose с профилями — ядро (Caddy, web, API, эмбеддер, PostgreSQL, Valkey, Mailpit), `fsp` (Keycloak и реестр), `ats` (демо-ATS), `obs` (Prometheus, Loki, Alloy, Grafana), `full` — всё. Лимиты памяти контейнеров

профиля `full` в сумме — 3 776 МиБ (из них Keycloak — 768, PostgreSQL — 512, API и эмбеддер — по 384), ядра — 1 984 МиБ; под нагрузкой ядро занимает заметно меньше лимитов (§12.4), весь стек укладывается в недорогой VPS.

3. Справочники

- **Направления и специализации:** разработка (бэкенд, фронтенд), инфраструктура (DevOps/SRE), качество (QA), данные и ИИ (ML/DS), безопасность.
- **Грейды:** Junior, Middle, Senior. Пороги на шкале способности θ: −1, 0, +1.
- **Навыки:** 83 навыка, 491 синоним на русском и английском, включая сленг («постгрес», «кубер», «джава»), 77 рёбер близости (PostgreSQL ~ MySQL, React ~ Vue…).

4. Механика тестирования

4.1. Идея

Единый набор заданий утекает; генерация нейросетью «под каждого» не гарантирует сопоставимую сложность. Мы разделили две задачи:

- **Неповторимость** даёт генератор: шаблон задания по случайному `seed` выдаёт новый экземпляр — другие числа, код, имена, порядок и набор вариантов. Правильный ответ **вычисляется** генератором (мини-симулятор event loop JavaScript, расчёт подсети, выполнение SQL на сгенерированной таблице и т. п.), а не хранится.
- **Сопоставимость** даёт калибровка: у каждого уровня шаблона есть параметры модели IRT — дискриминативность `a`, сложность `b`, угадывание `c`. Способность всех кандидатов измеряется на общей шкале θ, поэтому разные задания дают сравнимый результат.

Сложность внутри шаблона задают «радикалы» (например, число асинхронных задач в event loop), а «инциденталы» (числа, имена, порядок) меняются без влияния на сложность.

4.2. Банк заданий

62 шаблона: 52 вычислительных генератора и 10 вариативных пулов (5–7 формулировок, 3–4 равноправных правильных ответа и 7–10 дистракторов на каждый вопрос). У каждого шаблона 1–3 уровня сложности, $b \approx -1{,}4 \ldots +1{,}4$. Модульный тест закрепляет разнообразие: на 500 случайных `seed` каждый уровень каждого шаблона даёт не меньше 490 различимых экземпляров — при N равновероятных вариантах это значит, что N исчисляется десятками тысяч.

Специализация	Шаблонов	Тем	Примеры
Бэкенд	24	10	изменяемый аргумент по умолчанию в Python, порядок `defer` в Go, выбор индекса, token bucket, LRU, аномалии изоляции, SQL `GROUP BY / HAVING`
Фронтенд	15	6	порядок вывода в event loop, замыкания в цикле, приведение типов, специфичность CSS, box model, батчинг состояния React
DevOps	18	10	кэш слоёв Docker, ресурсы Kubernetes, CIDR, `location` в nginx, cron, права Linux, `increase()` в Prometheus, бюджет ошибок
QA	18	8	граничные значения, классы эквивалентности, pairwise, диаграммы состояний, таблицы решений, проверки API
ML	18	8	метрики по матрице ошибок, теорема Байеса, градиентный шаг, k-means, `groupby` в pandas, формы массивов NumPy
Безопасность	15	8	класс уязвимости во фрагменте кода, энтропия пароля, XOR-шифр, Base64, обход фильтров, правила межсетевого экрана

Типы ответа: выбор одного, выбор нескольких, число, текст. Задания и интерфейс — на русском, код — на соответствующем языке.

4.3. Адаптивный тест

Параметр	Значение
Модель	3PL; $c = 1/k$ для выбора одного из k вариантов, 1/31 для «выберите все верные» из пяти, 0 для числового и текстового ответа
Оценка θ	Апостериорное среднее (EAP), сетка из 81 точки на [−4; 4], априорное N(порог заявленного грейда; 1)
Выбор задания	Максимум информации Фишера в текущей θ с поправкой на баланс тем (темы с недобором получают приоритет), случайный выбор среди пяти лучших (контроль экспозиции), без повторов шаблона
Остановка	Не менее 8 и не более 20 заданий; стоп, когда 95-процентный интервал по θ не содержит порог заявленного грейда
Решение	$\hat{\theta} \geqslant$ порога $\longrightarrow$ тест сдан
«Уверенно сдал»	Измеренный грейд выше заявленного или $P(\theta \geqslant$ следующего порога$) \geqslant 0{,}9 \longrightarrow$ предлагается дотест на грейд выше
Время	Лимит на задание (60–180 с), просроченный ответ засчитывается как неверный
Аномалии	Статистика person-fit l_z («верно на трудных, неверно на лёгких», $l_z < -2$); вместе с событиями видеозаписи (уход со вкладки, вставка текста) результат помечается «требует подтверждения» — §4.7

Стек кандидата из опроса повышает приоритет шаблонов с совпадающими навыками (× 1,5), но шкала и пороги остаются общими. Если тест сдан, навыки верно решённых заданий получают отметку «подтверждён тестом».

4.4. Почему тест устойчив к распространению заданий

Что утекло	Что даёт утечка
Конкретный экземпляр с ответом	Практически ничего: следующий кандидат получит другой seed, а значит другой код и другой ответ
Шаблон («тут спрашивают порядок вывода event loop»)	Понимание, как решать такой класс задач, — ровно тот навык, который мы измеряем
Весь банк шаблонов	Ответы всё равно нужно вычислить, а на каждое задание действует лимит времени

Количественная проверка — §6.2: даже при базе из ~44 тыс. слитых экземпляров совпадения встречаются в 0,02–0,05 % заданий, и доля прошедших слабых кандидатов не растёт (9–10 %), тогда как у фиксированного теста — до 100 %.

Роль нейросетей: только офлайн — предложить новый шаблон, который проходит ревью и пробную калибровку (показывается кандидатам без влияния на оценку) до попадания в рабочий банк. Сопоставимость обеспечивает калибровка, а не генерация.

4.5. Правила смены грейда

- **Первичное размещение.** Кандидат выбирает грейд и проходит тест. Не сдал — сразу может пройти тест на грейд ниже, и так вплоть до Junior; повтор того же или более высокого грейда — через 30 дней (иначе тест можно было бы перебирать). Не сдал Junior — пересдача через 30 дней, категории пока нет.
- **Неудачная попытка закрывает свой грейд на 30 дней** — для повтора, дотеста, повышения и понижения.
- **Брошенный тест — тоже неудачная попытка.** Если 30 минут после дедлайна текущего задания ответа нет, тест закрывается и засчитывается как неудачная попытка на свой грейд — с моментом «дедлайн + 30 минут», когда бы кандидат ни вернулся. Иначе неудачно идущий тест можно было бы бросить и сразу пересдать. Сбоем платформы это не считается никогда: тест не ждёт ни камеру, ни загрузку записи, а «запись прервалась» лишь помечает результат.

- **Уверенно сдал** — один раз предлагается дотест на грейд выше, в том числе после уверенной сдачи на шаг ниже. Но если этот грейд кандидат провалил или бросил меньше 30 дней назад, дотест откроется только по истечении этих 30 дней: провалил Middle, уверенно сдал Junior — дотест на Middle через 30 дней после провала, а не сразу. До тех пор кабинет показывает его с датой.
- **После присвоения** смена грейда вверх или вниз — не чаще раза в 90 дней (настраивается). Неудачная попытка повышения не понижает грейд; следующая — через 30 дней, но не раньше, чем закончится 90-дневное окно от присвоения. Если действуют несколько ограничений, называется дата, когда снимется последнее.
- Кандидат видит текущую категорию, историю опросов, тестов и присвоений, доступные попытки, а у недоступных — дату и причину.

4.6. Как тест строится от вакансии

`GET /api/v1/catalog/test-plan` и `GET /api/v1/employer/needs/{id}/test-plan` показывают по описанию вакансии: целевой порог θ, темы, шаблоны, которые получат приоритет из-за стека вакансии, и примеры заданий для двух разных кандидатов. В демо-данных у работодателя `employer@demo.fsp` шесть потребностей — по одной на специализацию: Go-разработчик в платежи, фронтенд личного кабинета мерчанта, DevOps платформы процессинга, QA по автоматизации API выплат, ML-инженер антифрода и AppSec-инженер платёжного API. У каждой свой план теста («Как проверяется категория») и непустая подборка. Например, для «Middle Go-разработчик: PostgreSQL, Kafka» приоритет получают `sql.group_by_having`, `sql.left_join_rows`, `go.defer_order`, `db.index_choice`, `db.isolation_anomaly`. Кандидаты А и Б получают одни и те же шаблоны и уровни, но разные запросы и данные — `WHERE o.amount > 600` у одного и `> 750` у другого.

4.7. Видеозапись теста и журнал событий

Это наше дополнение: ТЗ прокторинга не требует. Цель — подтвердить, что тест пройден самостоятельно, не нарушив приватность и не мешая отвечать. Режим задаётся переменной `PROCTORING=optional|off`; по умолчанию камера предлагается, но не обязательна.

Перед тестом кандидат видит шаг «Видеозапись во время теста»: предпросмотр камеры и три коротких пункта — что записываем (только видео с камеры, без звука и без экрана, в низком качестве), кто увидит (сам кандидат и модератор при разборе спора; работодатель — только отметку) и сколько храним (90 дней в зашифрованном виде). Галочка согласия — отдельный версионированный документ в общей системе согласий. Две кнопки: «Начать с видеозаписью» и «Продолжить без камеры». Без камеры тест засчитывается так же, у результата отметка «без видеозаписи».

Во время теста браузер записывает видео 320×240 с низким битрейтом и каждые 10 секунд отправляет фрагмент на сервер; в углу — индикатор «Идёт запись» с миниатюрой камеры. Отдельно, с камерой и без, отмечаются события: вкладка скрыта, окно потеряло фокус, выход из полноэкранного режима (его предлагаем, но не навязываем), вставка текста в поле ответа, камера отключилась, запись остановилась. Отправка фоновая и с повторами; любой сбой превращается в «запись прервалась», тест при этом продолжается.

Итог — сводка с одним из статусов «с видеозаписью», «без видеозаписи», «запись прервалась» и числом событий каждого типа. Она пересчитывается из сохранённых данных при каждом чтении, поэтому учитывает последний фрагмент, который рекордер отдаёт уже после последнего ответа (сервер принимает его ещё 60 секунд).

«Требует подтверждения». Ни флаг person-fit, ни уход со вкладки по отдельности не доказывают помощь со стороны: порог $l_z < -2$ срабатывает примерно у 2 % честных кандидатов, а переключить вкладку может любой. Поэтому отметка ставится, только когда аномальный шаблон ответов совпал с уходами со вкладки, вставкой текста или выходом из полноэкранного режима. Грейд автоматически не меняется: отметку видят кандидат и модератор, который решает, что с ней делать (ниже).

Кто	Что видит
Кандидат	Сводку в результате и в истории тестов, свою запись, дату её удаления и ровно ту отметку, которую видят компании
Работодатель	Только отметку в анонимной карточке и профиле: «Тест с видеозаписью», «Тест без видеозаписи», «Запись прервалась», «Есть замечания: 2 ухода со вкладки». Видео недоступно ни до, ни после принятия приглашения
Модератор	Ответы по сложности, журнал событий и запись любой сессии — для разбора спорного результата; каждый просмотр записи — в журнале, а кандидат видит его дату

Разбор спорного результата. Отметку «требует подтверждения» разбирает модератор — роль платформы со своим кабинетом (`/admin`); регистрацией её не получить. Очередь показывает такие тесты, сначала самые старые; отдельно — тесты с уходами со вкладки и все тесты с видеозаписью. В карточке теста модератор видит категорию и грейд, анонимный код кандидата, оценку уровня с порогом, что этот тест сейчас делает с категорией, статистику согласованности ответов `l_z` с понятным разбором («задания сложнее уровня: верно 6 из 10, задания легче: неверно 4 из 8») и диаграммой верных и неверных ответов по сложности, запись и ленту «задания и события»: у каждого ухода со вкладки и вставки текста видно, какое задание было на экране. Имени, контактов и почты кандидата в карточке нет: чтобы решить, сам ли человек проходил тест, личность не нужна, а лишние персональные данные — лишний риск. Запись загружается только по кнопке, и каждый просмотр попадает в журнал.

Решений три, у каждого обязательный комментарий, который прочитает кандидат; решение одно и окончательное.

Решение	Что происходит
«Подтвердить результат»	Отметка снимается. Работодатели видят у категории «Результат подтверждён»
«Аннулировать результат»	Категория, которую дал тест, снимается, и возвращается прежняя — или категории не будет. Навыки, которые подтвердил только этот тест, теряют отметку «по тесту». Тест считается неудачной попыткой на свой грейд с моментом решения: этот грейд снова доступен через 30 дней после решения — медленный разбор не дарит немедленную пересдачу. Ограничение «смена грейда раз в 90 дней» снова считается от прежнего присвоения, а не от аннулированного
«Назначить пересдачу с видеозаписью»	Результат аннулируется так же, но ту же попытку можно пройти сразу и вне обычных сроков — только с камерой: тест на этот грейд и выше без камеры не начнётся, пока не будет завершён тест с видеозаписью. Брошенный тест требование не снимает, тест на грейд ниже — свободен

Правила записаны в той же чистой политике грейда, что и сроки пересдач (два новых исхода попытки: «аннулирована» и «назначена пересдача»), и покрыты модульными тестами: восстановление прежнего грейда и его окна, блок на 30 дней от решения, немедленная пересдача, отказ в старте без камеры, перевод статусов сессий и решений модератора в исходы политики. Путь модератора от очереди до решения, которое видят кандидат и работодатель, проверяет black-box тест (§6.6). Кандидат видит решение и комментарий в истории тестов, пометку «аннулировано» в истории категории и даты, когда модератор открывал его запись; работодатель — только обновлённую отметку.

5. Механика подбора и категоризация

5.1. Категоризация

Категория присваивается только по результату теста: сданный тест на заявленный грейд ⟶ категория «специализация · грейд». Самоописание и резюме влияют лишь на вторичные признаки (стек, ожидания) и никогда не определяют категорию. Кандидат без категории в поиске не участвует.

5.2. Сценарий выхода на контакт

1. Работодатель описывает потребность — формой или вставкой текста вакансии; система распознаёт специализацию, грейд, обязательные и желательные навыки, вилку, формат.
2. Система показывает **рекомендованные категории**: сколько кандидатов, сколько с ФСП, медианные ожидания, насколько категория покрывает стек.
3. **Подборка** — анонимные карточки: категория, сила результата теста, навыки с отметкой «подтверждён тестом / подтверждён ФСП / из резюме / заявлен», достижения ФСП, опыт (названия компаний — по желанию кандидата), ожидания. Без имени и контактов.
4. **Объяснение** у каждого кандидата: балл 0–100, разложение по признакам, причины («Все обязательные навыки: Go, PostgreSQL, Kafka; 1 подтверждён тестом», «Результат теста выше 71 % диапазона грейда Middle», «Алгоритмы — подтверждено ФСП», «ФСП: Мастер спорта по продуктовому программированию, 2023») и пробелы («Нет обязательного навыка: Kubernetes», «Вместо PostgreSQL — близкий навык MySQL», «Навык Docker только заявлен»).
5. **Приглашение**: что предлагаем, сообщение, вилка «от–до» (обязательно), формат, способ связи с компанией; можно без вакансии.
6. Кандидат видит условия до общения и принимает или отклоняет с причиной (зарплата, стек, формат, компания, не ищу работу).
7. После принятия работодатель получает контакты; каждое обращение к контактам записывается в журнал.

5.3. Ранжирование

Жёсткие фильтры: кандидат видим и открыт к предложениям, категория нужной специализации, грейд из запрошенных (или соседний, если разрешено), формат и город совместимы.

Балл — взвешенная сумма признаков из [0; 1], веса подобраны на валидации (§6.3):

Признак	Вес	Расчёт
Обязательные навыки	0,41	Среднее по навыкам потребности: близость × доверие к источнику (тест или результат ФСП — 1,0, §7.4; резюме — 0,7; заявлен — 0,5)
Грейд	0,15	1,0 — точное совпадение, 0,6 — на грейд выше, 0,5 — на грейд ниже
Результат теста	0,10	Положение θ внутри диапазона грейда
Достижения ФСП	0,10	ФСП-скор (§7.3)
Желательные навыки	0,08	Как обязательные
Зарплата	0,07	1,0, если ожидания в вилке; линейно до 0 при превышении на 30 %; ожидания не указаны — нейтральные 0,7
Актуальность	0,05	exp(−дней с последней активности / 60): тест, задание, правка профиля, опрос, синхронизация ФСП, отклик, ответ на приглашение
Близость описаний	0,04	По смыслу — косинус эмбеддингов описания потребности и профиля (роли, «о себе», места работы), переведённый в 0–1 калибровкой (§5.6); пока вектора нет — полнотекстовая релевантность «о себе» описанию потребности (русская морфология); если в потребности указаны отрасли — 0,6 · текст + 0,4 · совпадение отраслей опыта

Признаки без данных (нет желательных навыков, нет описания и отраслей) выпадают, веса перенормируются. Без потребности — при просмотре категории — кандидаты упорядочены по силе профиля: 0,6 · тест + 0,25 · ФСП + 0,15 · актуальность.

Подбор 10 000 кандидатов с объяснением топ-20 занимает около 0,3 мс; в ранжирующем цикле нет аллокаций, тексты объяснений собираются только для итоговых кандидатов. Векторы профилей лежат в том же снимке сжатыми до байта на число, близость по смыслу всех кандидатов к потребности считается один раз на снимок (§12.4). Пул специализации API держит в памяти и сбрасывает при каждом изменении, влияющем на выдачу:

при 10 000 кандидатах и 50 одновременных клиентах поиск по банку под потребность отвечает за 43 мс (p95) — §12.4.

5.4. Уточнение без потери подборки

Подборка хранится. При уточнении фильтров она пересчитывается, но закреплённые кандидаты остаются (с пометкой, если не проходят новые фильтры), скрытые не возвращаются, а новые помечаются как «новые».

5.5. Чем это отличается от отклика на вакансию

- Инициатива у работодателя, который ищет по подтверждённым тестом категориям, а не по ключевым словам.
- Условия — в первом же сообщении: кандидат решает, тратить ли время.
- **Квота**: до 30 приглашений компании за 7 дней — сколько именно, зависит от уровня доверия (§10.1): новая компания начинает с 10. Плюс одно действующее приглашение на пару «компания–кандидат». Работодатель выбирает точечно, кандидат не тонет в спаме.
- Приглашение живёт 14 дней. Работодатель видит воронку и агрегированные причины отказов, кандидат — анонимную статистику спроса в своей категории: сколько компаний смотрели профиль, медианная вилка приглашений.
- Классический отклик остаётся дополнительным путём: контакты раскрываются и при собственном отклике.

5.6. Машинное обучение: смысловая близость текстов

Словарь навыков с синонимами и правила разбора понимают названия технологий, но не смысл фраз: «писал консьюмеры потока событий» не содержит слова Kafka, а пересказанная своими словами чужая вакансия не делит с оригиналом ни одной пары слов. Для этого в решении есть ML-компонент — эмбеддинги предложений: модель переводит текст в вектор из 384 чисел, и тексты о близком оказываются рядом.

Модель и почему локальная. `intfloat/multilingual-e5-small` (лицензия MIT, 118 млн параметров, русский и английский) работает в отдельном сервисе `embedder` на нашем же сервере, на CPU. Тексты профилей и вакансий — персональные и коммерческие данные; внешний API отправлял бы их третьей стороне, а по 152-ФЗ и здравому смыслу они не должны покидать инфраструктуру. Сервис написан на Rust: модель исполняет candle, текст режет `tokenizers` — без Python и без C-зависимостей. Векторы совпадают с эталонной реализацией (transformers) до 10^{-7}, это проверяют тесты.

Выбор модели — на наших данных (`docs/validation/embeddings.md`, сравнение воспроизводится `validate models`):

	rubert-tiny2 (MIT, 29 млн)	multilingual-e5-small (MIT, 118 млн)
Связь косинуса с реальными навыками кандидата (корреляция)	−0,08	0,30
Только по смыслу: доля релевантных в топ-10 (случайно — 0,22)	0,17	0,28
Пересказ вакансии против разных вакансий, ROC AUC	0,948	0,978
Пересказы, пойманные без ложных срабатываний	25 %	50 %
Время на профиль (~100 токенов), один поток / два	7 / 5 мс	70 / 48 мс
Память сервиса после загрузки / под нагрузкой (на M5, с нашей загрузкой весов)	~40 / ~90 МБ	~130 / ~190 МБ

Мы склонялись к маленькой модели — цель недорогой VPS. Но rubert-tiny2 по своей карточке берёт вектор первого токена, и на длинных текстах профилей он следует за длиной текста, а не за содержанием; e5 явно лучше в обеих задачах. Её цена — время: профиль считается за 50–90 мс на двух потоках, но запросы модель не ждут, а 10 000 профилей демо-нагрузки встают в очередь и считаются за 7 минут. Память мы сократили сами: стандартная загрузка e5 занимает ~600 МБ, потому что 82 % весов — таблица векторов 250 тысяч токенов всех языков, а словарь токенизатора в библиотеке хранится деревом с хэш-таблицей в каждом узле (~390 МБ). Таблицу мы читаем прямо из отображённого в память файла весов (с диска подтягиваются только строки встреченных токенов), а словарь держим одной хэш-таблицей — разбиение на токены побайтно то же, проверено на 3 000 случайных строк. Итог

— ~110–130 МБ после загрузки и до ~250 МБ под нагрузкой, из них своей памяти ~180 МБ; лимит контейнера — 384 МБ.

Где работает.

Где	Что делает	Без эмбеддингов
Подбор, признак «Близость описаний»	Косинус векторов описания потребности и профиля (роли, «о себе», должности и описания мест работы), переведённый в 0–100 % по калибровке на обучающих потребностях. В объяснении — «Близость описаний по смыслу: 72 %», от 60 % — причина «Опыт близок к задаче по смыслу»	Полнотекстовая релевантность «о себе» (русская морфология), «по ключевым словам»
Антифейк	Опубликованная вакансия сравнивается с вакансиями других компаний за 90 дней; косинус ⩾ 0,953 — отметка «похоже на пересказ» в очереди модератора с оригиналом и числами, работодателю — примечание без названия чужой компании (§10.1)	Только MinHash: дословные копии и правки
«Похожие вакансии»	На странице вакансии и в кабинете соискателя — ближайшие по смыслу опубликованные, сначала той же специализации	Та же специализация, больше общих навыков
Проверка на своих данных (§6.4)	Близость резюме и вакансии по смыслу, векторы сохраняются рядом с отчётом	По ключевым словам, отчёт говорит почему

Векторы считаются после записи: сохранение профиля, потребности или вакансии ставит её в очередь в той же транзакции (триггер в базе — ни один путь записи её не забудет), фоновый воркер API раз в секунду забирает до 32 текстов, отправляет в модель только изменившиеся и пишет векторы в PostgreSQL (pgvector). Запрос пользователя модель не ждёт никогда. Векторы профилей лежат в снимке пула кандидатов в памяти API, сжатые до байта на число (388 байт на профиль); близость всех кандидатов к потребности считается один раз на снимок. `EMBEDDINGS=off` или остановленный сервис ничего не ломают: посчитанные векторы продолжают работать, новые тексты ждут, а без векторов каждое место работает как раньше — это проверяет отдельный black-box сценарий (§6.6).

Что показала валидация (§6.3, `docs/validation/matching.md`). Чтобы смысловой сигнал вообще было на чём проверить, генератор синтетики научился писать тексты: описания мест работы пересказывают задачи по *реальным* навыкам кандидата, а «о себе» перечисляет заявленный стек с тем же приукрашиванием. Сигнал в таких текстах есть — косинус связан с реальным покрытием обязательных навыков (корреляция 0,30), — но тест и подтверждённые навыки уже несут почти всё то же: на отложенных потребностях подбор с близостью по смыслу, по ключевым словам и совсем без неё неразличим (nDCG@10 0,838, 0,839 и 0,835 при погрешности ±0,02), и даже без заявленных навыков у кандидатов прибавки нет. Вес признака перебирался от 0,04 до 0,20 — всё, что больше 0,04, ухудшает подбор на обучающих потребностях, поэтому вес оставлен 0,04. На примере §6.4 из пяти вакансий смысловая близость немного переставила почти равных кандидатов в двух вакансиях, и метрики там ниже, чем с ключевыми словами (nDCG@10 0,92 против 0,96, в пределах погрешности). Это мы и говорим прямо: ценность модели в подборе — объяснимый сигнал по смыслу, а не рост метрик; заметный выигрыш — в антифейке, где MinHash не поймал ни одного из 12 пересказов, а эмбеддинги поймали половину без единого ложного срабатывания на калибровочном наборе (`docs/validation/paraphrase.md`). Подсказывать навык словаря по смыслу для незнакомых фраз резюме мы проверили и отказались: на 40 фразах с известным ответом ближайший навык верен в 11 случаях (28 %), точность даже при строгом пороге не выше 0,33.

6. Процедура валидации и результаты

Валидация — отдельный исполняемый модуль `crates/validate` (`make validate`), который вызывает **те же крейты**, что и API, и пишет отчёты в `docs/validation/`. Данных и разметки организаторы не дают, поэтому используется синтетика со скрытой «истиной».

6.1. Тестирование: точность и стабильность

Каждая сессия — настоящий движок; синтетические только ответы: вероятность верного ответа по 3PL при истинной $\theta \sim N(0; 1)$. 400 человек на специализацию, процедура как в API и по той же политике грейда `GradePolicy` (заявленный грейд → при провале сразу грейд ниже, вплоть до Junior). Пересдача — вторая независимая процедура с другими заданиями.

Сценарий	Решение «выше/ниже порога» верно	Итоговый грейд = истинный	Отклонение ⩽ 1 грейда	Совпадение при пересдаче	κ Коэна	RMSE θ	Заданий
Честная заявка, точная калибровка	87,7 %	87,0 %	100 %	81,1 %	0,74	0,35	13,6
20 % завышают, 10 % занижают грейд	88,0 %	81,2 %	100 %	80,2 %	0,73	0,35	13,5
Ошибка калибровки заданий (a ×0,8…1,2, b ±0,3)	87,1 %	86,2 %	100 %	80,2 %	0,72	0,35	13,6

Ошибки только «вниз» и только у границы грейда: тест на заявленный грейд не повышает принудительно, а пограничный кандидат может пересдать на грейд выше. Неточная калибровка почти не ухудшает результат.

Дискриминативность: точечно-бисериальная корреляция верного ответа с истинной θ — в среднем 0,30, у 94 % шаблонов ⩾ 0,2. Самые слабые шаблоны выводятся в отчёт как кандидаты на перекалибровку. Низкая корреляция у шаблона с несколькими уровнями может быть артефактом смешения уровней: адаптивный тест показывает трудный уровень более сильным кандидатам, поэтому внутри уровня корреляция выше, чем по шаблону в целом (так было с `go.defer_order`: 0,30 / 0,21 / 0,50 по уровням при 0,13 в сумме; уровень перекалиброван).

6.2. Тестирование: устойчивость к утечке

Слабые кандидаты (истинная θ от −0,9 до −0,1, ниже порога Middle) сдают тест на Middle.

Утекло	Фиксированный тест: прошли	Наш тест: прошли
0 %	9,4 %	9,7 %
25 % ответов	65,2 %	9,6 %
50 % ответов	99,9 %	9,3 %
100 % ответов	100 %	10,4 %

Фиксированный тест — публичная форма из 15 самых информативных заданий. Для нашего теста «утечка» — база из всех экземпляров, показанных в 600 предыдущих сессиях на специализацию (44 306 экземпляров); слабый кандидат отвечает верно, если ему выпал экземпляр из базы. Совпадения с базой случаются в 0,02–0,05 % заданий, поэтому утечка не даёт преимущества. Статистика person-fit ловит 11–12 % «сливщиков» фиксированного теста при 0,9 % ложных срабатываний.

6.3. Подбор: доля релевантных в топе

2 400 синтетических кандидатов, 120 потребностей для подбора весов и 120 отложенных для оценки. У кандидата есть скрытая истина (реальные навыки и θ) и наблюдаемый профиль с шумом: ~18 % заявленных навыков приукрашены, ~15 % реальных не заявлены, категория — из теста с ошибкой измерения, часть навыков подтверждена тестом. Релевантность (0–3) считается по истине функцией, отличной от скоринга. Релевантным считается оценка ⩾ 2.

С появлением эмбеддингов генератор пишет и тексты — иначе смысловому признаку нечего было бы проверять. Описания мест работы пересказывают задачи по *реальным* навыкам (каждый с вероятностью 0,75), чаще всего без названия технологии в том виде, в каком его пишет вакансия; «о себе» перечисляет заявленный стек с тем же приукрашиванием; описание потребности пересказывает её навыки своими словами; отрасли сторон выбираются независимо и для любого текстового метода — шум. Тексты идут из отдельного потока случайных чисел, поэтому

кандидаты и потребности — те же, что раньше, и строки без текстового признака совпадают с прежними цифрами. Это допущения генератора, а не свойство рынка: они показывают, что даёт смысловая близость, когда описание опыта отражает реальную работу.

Ранжирование	Precision@10	nDCG@10	MAP	MRR
Наш подбор (веса после валидации, близость описаний по смыслу)	0,941 ± 0,018	0,838 ± 0,022	0,624 ± 0,014	1,000
Наш подбор, исходные веса	0,898 ± 0,022	0,765 ± 0,027	0,596 ± 0,014	0,992
Ключевые слова резюме + заявленный стаж	0,425 ± 0,030	0,356 ± 0,028	0,275 ± 0,012	0,632
Только близость описаний по смыслу (семантический поиск)	0,283 ± 0,030	0,239 ± 0,023	0,263 ± 0,015	0,507
Случайный порядок в специализации	0,223 ± 0,025	0,173 ± 0,018	0,224 ± 0,015	0,414

MAP — средняя точность по всем релевантным кандидатам специализации (кандидаты, отсечённые фильтрами, считаются ненайденными), MRR — по первой десятке.

С эмбеддингами и без — те же веса, те же отложенные потребности, меняется только источник признака «Близость описаний»:

Близость описаний	Precision@10	nDCG@10	MAP	MRR
По смыслу (эмбеддинги multilingual-e5-small)	0,941 ± 0,018	0,838 ± 0,022	0,624 ± 0,014	1,000
По ключевым словам (эмбеддер выключен)	0,942 ± 0,017	0,839 ± 0,023	0,624 ± 0,014	0,996
Признак не применяется (описания нет)	0,941 ± 0,017	0,835 ± 0,023	0,622 ± 0,015	1,000
Профили без заявленных навыков: по смыслу	0,903 ± 0,022	0,796 ± 0,025	0,589 ± 0,014	1,000
Профили без заявленных навыков: по ключевым словам	0,905 ± 0,021	0,798 ± 0,025	0,589 ± 0,014	0,992
Профили без заявленных навыков: признак не применяется	0,906 ± 0,022	0,794 ± 0,025	0,587 ± 0,014	1,000

Прямо: на этих данных эмбеддинги метрики подбора не улучшают. Сигнал в текстах есть — косинус связан с тем, сколькими обязательными навыками кандидат владеет на самом деле (корреляция 0,30, у пересечения ключевых слов 0,29), а одна только смысловая близость ранжирует лучше случайного порядка. Но заявленные и подтверждённые тестом навыки, категория и θ уже несут почти всё то же самое, и разница между тремя источниками меньше погрешности — даже у кандидатов, которые не заполнили список навыков (на них веса не подбирались). Поэтому вес признака оставлен 0,04: подбор весов перебирал 0,04–0,20, и всё больше 0,04 ухудшало nDCG@10 на обучающих потребностях (0,837 → 0,834 при 0,08, 0,793 при 0,20). Калибровка близости по смыслу (косинус 0,88 → 0 %, 0,918 → 100 %) — 10-й и 99-й перцентили косинусов на обучающих потребностях.

Абляции: без результата теста nDCG падает до 0,826, без графа близости навыков — до 0,830, только по навыкам — до 0,739. Исходные веса (обязательные навыки 0,30, тест 0,25) проигрывали; на трёх случайных сидах устойчиво лучшим оказался вариант с большим весом стека и грейда — он и принят.

Модели (`docs/validation/embeddings.md`): на тех же данных rubert-tiny2 даёт полному подбору то же (nDCG@10 0,835), но её косинус не связан с реальными навыками (−0,08), а одна только её близость ранжирует хуже случайного (P@10 0,17); у multilingual-e5-small — 0,30 и 0,28. **Пересказы вакансий** (`docs/validation/paraphrase.md`): у 12 пересказов демо-вакансий косинус с оригиналом 0,930–0,973, у самой похожей «не копии» той же роли и стека — 0,953, у разных вакансий одной специализации — не выше 0,923; ROC AUC 0,978. MinHash не отмечает ни одного пересказа (у всех меньше 0,6), эмбеддинги при пороге 0,953 — шесть из двенадцати без ложных срабатываний.

6.4. Проверка на своих данных

Главный критерий подбора — доля релевантных кандидатов в топе на наборе пар «вакансия – кандидат». Вводить такой набор через интерфейс долго, поэтому есть пакетный прогон: ваши вакансии, резюме и разметка проходят через тот же разбор текстов, тот же скоринг и ту же базовую линию, что в §6.3. Командная строка и метод API принимают один формат и возвращают один отчёт.

Входные данные — JSON с тремя списками или каталог с файлами `vacancies.csv`, `candidates.csv` и `labels.csv`. В CSV разделитель — запятая или точка с запятой, многострочные тексты берутся в кавычки, списки внутри ячейки разделяются `|`.

Список	Обязательно	Необязательно — если нет, распознаётся из текста
`vacancies`	`id`, `text` — текст вакансии, заголовок в первой строке	`specialization`, `grades`, `must_skills`, `nice_skills`, `salary_min`, `salary_max`, `work_formats`, `city`, `allow_adjacent_grades`
`candidates`	`id`, `resume_text` — текст резюме	`specialization`, `grade`, `skills`, `salary_expectation`, `work_formats`, `city`, `relocation`, `test_result` (0–100), `fsp_achievements` (модель §7.2)
`labels`	`vacancy_id`, `candidate_id`, `relevance` — 0–3	—

Значения справочников — слаги, как в `GET /api/v1/catalog`: `backend`, `middle`, `remote`. Лимиты прогона: до 200 вакансий, 5 000 кандидатов и 100 000 оценок, тексты до 20 000 символов. Пример — `docs/validation/pairs-example/`: 5 вакансий, 60 резюме и 65 оценок в JSON и CSV, рядом готовый отчёт.

Чего нет во входных данных, то не выдумывается, а допущения видны в отчёте:

- Специализация и грейд кандидата, если их нет, берутся из разбора резюме — того же, что при загрузке PDF, — и помечаются «из резюме, не подтверждена тестом». Грейд из резюме — заявленный или по стажу; если его нет нигде, принимается Middle с пометкой. Кандидат без распознанной специализации остаётся без категории и в наш подбор не попадает, базовая линия его видит.
- Без результата теста берётся нейтральное значение — середина диапазона грейда, как у типичного кандидата категории: отсутствие теста не помогает и не мешает. В объяснении написано «Результата теста нет», а не «тест пройден». `test_result` из входных данных — положение результата внутри грейда, 0–100.
- Соседний грейд по умолчанию допускается со штрафом признака «Грейд»: грейд из резюме не подтверждён тестом, и жёсткий фильтр по нему отсекал бы подходящих. Поле `allow_adjacent_grades` это меняет.
- Активности нет, поэтому признак «Актуальность» у всех нулевой и на порядок не влияет. Близость описаний — по смыслу, как в API: косинус эмбеддингов текста вакансии и резюме той же моделью и с той же калибровкой. CLI считает их в процессе (модель — `make model`), API — через сервис эмбеддингов, если в наборе не больше 300 текстов. Без модели — доля основ из первых 40 значимых слов вакансии, найденных в резюме, нормированная на лучшего кандидата специализации, как ключевые слова в API; отчёт пишет, какой способ сработал и почему (поле `text`). Векторы, с которыми посчитан отчёт, CLI кладёт рядом (`report.vectors.json`): по ним отчёт воспроизводится без модели.

Отчёт. Для каждой вакансии: как она распознана, топ-k нашего подбора с баллом, причинами, пробелами и источником категории, топ-k базовой линии «ключевые слова резюме + заявленный стаж» — той же, что в §6.3. При разметке считаются Precision@5, Precision@k, nDCG@k, MAP и MRR по каждой вакансии и в среднем по вакансиям, где есть хотя бы один релевантный кандидат. Релевантен кандидат с оценкой ⩾ 2, неразмеченная пара считается нерелевантной, доля размеченных в топе выводится рядом. Блок покрытия показывает, сколько полей взято из данных и сколько распознано, у скольких кандидатов нет теста и какие вакансии разобраны неуверенно.

Запуск из командной строки — отчёт в Markdown и JSON:

```
cargo run -p validate --release -- pairs --input data.json --out report.md
cargo run -p validate --release -- pairs --input каталог_с_csv --k 10 --now 2026-10-01
```

Или из Swagger (`/api/docs`, раздел «Валидация»): `POST /api/v1/eval/pairs` с тем же JSON, пример запроса уже подставлен. Метод публичный и ничего не сохраняет; тело до 8 МиБ, не больше 10 запросов за 10 минут с одного IP. Полный по лимитам набор — 200 вакансий и 5 000 резюме — командная строка обрабатывает за секунды.

На примере (разметка составлена до первого прогона):

Ранжирование	Precision@5	Precision@10	nDCG@10	MAP	MRR
Наш подбор (близость описаний по смыслу)	0,72	0,42	0,92	0,89	0,90
Наш подбор, близость по ключевым словам (без модели)	0,76	0,42	0,96	0,93	1,00
Ключевые слова резюме + заявленный стаж	0,68	0,42	0,85	0,82	0,80

Precision@10 одинакова: релевантных на вакансию 3–6, и все способы поднимают их в первую десятку; разница — в порядке внутри топа. С близостью по смыслу в двух вакансиях из пяти поменялись местами почти равные кандидаты (баллы отличались на 0,1–1 из 100), и метрики стали ниже, чем с ключевыми словами; на пяти вакансиях это в пределах погрешности, а разметку мы не подгоняем. Пример показывает и предел работы без тестов: выпускник курсов с полным списком ключевых слов стоит высоко в обоих ранжированиях, потому что без теста система видит только слова резюме. В рабочем режиме категорию и силу результата даёт тест.

6.5. Ограничения и экспертная проверка

Синтетическая «истина» задаётся командой, поэтому абсолютные цифры описывают поведение системы в заданных предпосылках, а не рынок. Сравнение с базовыми линиями и абляции от этого не зависят. Следующий шаг — экспертная разметка: 10–20 реальных вакансий партнёров ФСП, топ-20 выдачи каждой оценивают два эксперта по шкале 0–3; Precision@10 и nDCG@10 по такой разметке считает прогон из §6.4. Для тестов — пилот на участниках соревнований ФСП: сравнение грейда по тесту с разрядом и результатами соревнований, повторная сдача через неделю.

6.6. Корректность кода: модульные и black-box тесты

Валидация выше отвечает на вопрос «хороши ли механики». Корректность реализации проверяют тесты двух видов — и только они: без интеграционных тестов, которые лезут в базу в обход API.

Модульные (`make test`, 554 теста, секунды, без базы и сети). Чистые крейты `assessment`, `matching`, `catalog` и чистые функции API рядом с хендлерами: хендлер собирает факты из базы, решение принимает функция, которую можно проверить без базы. Так покрыты политика грейда (провал, брошенный тест, аннулирование и назначенная пересдача, требование камеры, перевод статусов сессий в исходы), права решений модератора, возраст и согласие законного представителя, условия публикации профиля и выдачи согласий, расписание писем (сбор, пауза, дневной лимит, приглашение вне паузы), состав полей уведомлений без ПДн, отписка, данные для ATS до и после принятия, подписи вебхуков и событий реестра с окном свежести, разбор вакансии из ATS, доверие и квота, поколения кешей, курсоры, ключ идемпотентности, выгрузка по ст. 14. Эмбеддинги: пулинг и нормировка, квантование и косинус, калибровка, план проходов модели, компактный токенизатор, очередь и повторы, отметка пересказа, соседи вакансии, тексты объяснения; совпадение векторов с эталонной реализацией и разбиения на токены с библиотечным — тесты с весами модели (`cargo test -p embedder -- --ignored` после `make model`).

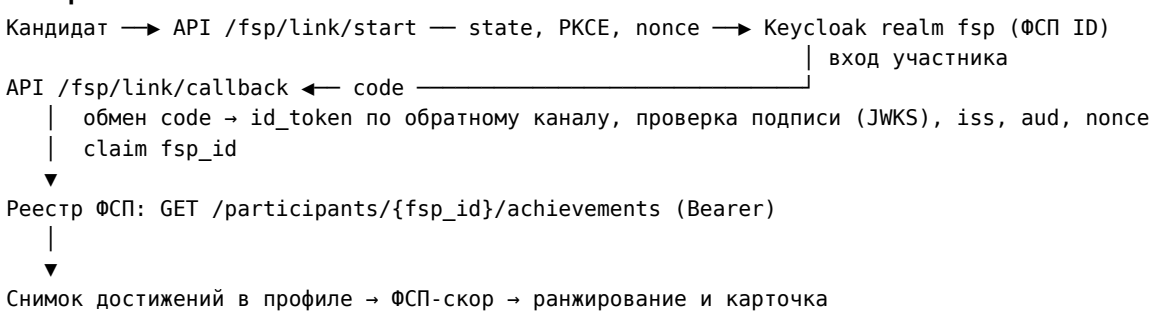
Black-box (`make blackbox`, Playwright, 15 файлов сценариев и отдельный `make blackbox-degraded`). Свежий стек в Docker с демо-данными; тесты видят систему так, как её видят люди и внешние системы: сайт и API через :8080, почта в Mailpit, демо-ATS, админка реестра ФСП. Ни SQL, ни доступа в контейнеры, ни правильных ответов теста — новый соискатель отвечает как придётся, и проверяется, что исход согласован: категория присвоена или ясно сказано «не подтверждён» и что делать дальше.

Что проверяется	Сценарии
Обязательный сценарий ТЗ	S-1 регистрация с подтверждением из письма; S-2 опрос, шаг видеозаписи с камерой и адаптивный тест до результата; S-3 категория; S-4 потребность → категории → подборка с «Как посчитан балл» → «Как проверяется категория»; S-5 приглашение → принятие → контакты; S-6 отклик на вакансию
Видеозапись	Соискатель смотрит свою запись, работодателю видео недоступно (403), в карточке — только отметка
152-ФЗ	Отзыв согласия на публикацию скрывает из поиска; без согласия видимость не включить (409 и диалог); повторное согласие возвращает; «Кто открывал ваши контакты»; выгрузка данных; разделы `/privacy`; одновременные регистрации на один email
14–17 лет	Отказ младше 14; письмо представителю после подтверждения почты; демо-подросток скрыт до согласия, «Отправить письмо ещё раз» → согласие по ссылке → в поиске с отметкой «14–17 лет», «Только 18+» скрывает
Модерация	Очередь, аннулирование с комментарием (соискатель видит решение и уведомление), подтверждение («Результат подтверждён» у работодателя), журнал, 403 на всех операциях модерации для остальных ролей, отклонённая жалоба на странице доверия компании без автора
Уведомления	Письмо о приглашении с вилкой, сроком и `List-Unsubscribe`, значок; принятие — уведомление и письмо компании без ПДн; отписка в один клик
ATS и реестр ФСП	Вебхук и API-ключ в «Интеграциях»; подписанные события в демо-ATS, контакты только в `invitation.accepted`; «Нанят» из ATS у соискателя; опрос по курсору; идемпотентный импорт; push реестра → достижение и уведомление без ручной синхронизации; подписи событий реестра
Доверие и API	Страница доверия новой компании, предупреждение о вилке, похожая вакансия, двойная отправка с `Idempotency-Key`; problem+json, `Location`, кеширование с ETag/304, `POST /eval/pairs` воспроизводит отчёт §6.4
Сайт	200 и один `h1`, `robots.txt`, `sitemap.xml`, `llms.txt`, JSON-LD с `JobPosting`; axe без нарушений и без горизонтальной прокрутки на 375 px на публичных страницах и в кабинетах
Эмбеддинги	«Похожие вакансии» на сайте и в кабинете соискателя: соседи по смыслу, сначала той же специализации; в разложении балла подборки — «Близость описаний по смыслу»; пересказ чужой вакансии новой компанией: MinHash молчит, отметка по смыслу — работодателю без имени оригинала, модератору с ним, квота не снижается. Отдельно (`make blackbox-degraded`, эмбеддер останавливает Makefile, не тест): публикация, похожие вакансии по навыкам, подборка «по ключевым словам» и `POST /eval/pairs` с причиной работают

Что не покрыто и почему. Откат изменения при сбое записи уведомления, события outbox или решения модератора — нужна инъекция отказа в базу. Гонки и блокировки (два обхода напоминаний, захват событий репликами с `SKIP LOCKED`, горизонт опроса ATS по открытым транзакциям, гонка регистраций за уникальный индекс — снаружи виден только итог) — поведение базы, а не кода. Распространение сброса кеша между репликами через Valkey — в стеке одна реплика. Всё, что зависит от часов: брошенный тест через 30 минут, 18-летие и очистка данных представителя, истечение приглашений, срок хранения записей — времени снаружи не подвинуть, правила покрыты модульными тестами на заданном `now`.

7. Интеграция с ФСП

7.1. Привязка ФСП ID



- ФСП ID работает на Keycloak; совместимость — стандартный OpenID Connect, клиент `fsp-talent` с обязательным PKCE S256. Claim `fsp_id` — атрибут пользователя Keycloak.
- Один ФСП ID можно привязать только к одному профилю. Отвязка удаляет снимок достижений и навыки, которые он подтверждал (§7.4).
- Если реестр недоступен, привязка сохраняется, а синхронизацию можно повторить из настроек. Достижения обновляются при привязке, по кнопке и по событию от реестра (§7.5).
- Для интеграции с реальным реестром нужны его адрес и токен доступа. Сейчас API передаёт заглушке статический Bearer-токен (`FSP_REGISTRY_TOKEN`); получать токен у Keycloak по client credentials или token exchange — небольшая доработка клиента реестра.

7.2. Какие данные получаем

Для каждого достижения: тип (разряд или соревнование), дисциплина (алгоритмическое, продуктовое программирование, программирование систем ИБ, систем ИИ, робототехники, БАС), разряд (юношеский, III–I, КМС, МС), уровень соревнования (региональное, межрегиональное, всероссийское, международное), место (1–3, финалист, участник), команда, роль, дата, ссылка на протокол.

7.3. ФСП-скор

Вклад достижения = вес уровня × вес места (или вес разряда) × релевантность дисциплины специализации × затухание с периодом полураспада 2 года. Итог — «шумное ИЛИ»: $1 - \prod(1 - v_i)$. Скор растёт с каждым достижением, но насыщается и не может перевесить тест и стек (вес в подборе — 0,10). Учитывается только высший разряд в дисциплине.

Членство в ФСП не учитывается. Организаторы уточнили: членство платное и для разряда не нужно, поэтому в анкете его нет, и в скор оно не входит. Считаются только разряды и результаты соревнований из реестра; привязанный ФСП ID без достижений ничего не меняет.

Кандидат без истории ФСП получает 0 без штрафа: в карточке нейтральное «Нет данных ФСП», отсутствие ФСП никогда не попадает в «пробелы». Кандидат может скрыть ФСП-данные — тогда они не участвуют и в ранжировании, вместе с навыками из ФСП.

7.4. Навыки, подтверждённые ФСП

Результат ФСП подтверждает навыки так же, как тест: доверие 1,0 вместо 0,5 у заявленного навыка, и работодатель видит это в объяснении — «Алгоритмы — подтверждено ФСП». Реестр хранит дисциплину и результат, но не стек команды и не роль участника, поэтому правило консервативное: навык засчитывается, только если его упражняет любое соревнование дисциплины.

Дисциплина	Подтверждённые навыки
Алгоритмическое программирование	Алгоритмы, Структуры данных
Программирование систем ИБ	Пентест (CTF — поиск и эксплуатация уязвимостей)
Программирование систем ИИ	Машинное обучение, Метрики качества ML (модель на скрытых данных, место по метрике)
Продуктовое программирование	— (стек продукта неизвестен; результат и так весит в ФСП-скоре)
Робототехника, БАС	— (в словаре нет навыков встраиваемых систем)

- Считается только результат: разряд или место 1–3 и финал; участие ничего не подтверждает.
- Навыки появляются при привязке и синхронизации и удаляются при отвязке — вместе со снимком достижений, в одной транзакции. Навыки из других источников не трогаются: подтверждённый и тестом, и ФСП навык в объяснении назван подтверждённым тестом.
- Скрытые ФСП-данные скрывают и эти навыки: они не участвуют ни в ранжировании, ни в объяснении.
- Правило — одна чистая функция с тестами; ей же пользуется проверка на своих данных (§6.4) для поля `fsp_achievements`.

7.5. Обновления из реестра: pull и push

Достижения попадают в профиль двумя путями. **Pull** — платформа сама спрашивает реестр: при привязке ФСП ID и по кнопке «Обновить из реестра». **Push** — реестр сообщает, что у участника что-то изменилось, и платформа тут же забирает новые данные: соискатель видит новый результат, не заходя в настройки, а компании — в карточке.

```
PULL (привязка, «Обновить из реестра»)
Соискатель ⟶ API: POST /fsp/sync
                API ⟶ Реестр: GET /participants/{fsp_id}/achievements (Bearer)
                API ⟵ достижения
                транзакция: снимок + навыки, подтверждённые ФСП → сброс кеша подбора
Соискатель ⟵ обновлённый список

PUSH (новый результат в реестре)
Реестр ⟶ API: POST /fsp/registry/events {event_id, type: achievements.changed, fsp_id, occurred_at}
              FSP-Signature: t=…,v1=HMAC-SHA256(общий секрет, t + "." + тело)
          API: лимит → подпись и метка времени (±5 минут) → fsp_id привязан? event_id новый?
          API ⟶ Реестр: GET /participants/{fsp_id}/achievements (Bearer) — та же синхронизация, что
в pull
          транзакция: событие, снимок, навыки, уведомление «Новые достижения ФСП в профиле»
          после коммита — сброс кеша подбора, если что-то изменилось
Реестр ⟵ 202
```

- **В событии нет данных** — только идентификаторы; достижения платформа забирает по своему токену. Даже поддельное событие не может ничего записать в профиль — только заставить лишний раз прочитать реестр, а это ограничено лимитом.
- **Подпись** — та же схема, что у наших вебхуков для ATS (§10.2): HMAC-SHA256 с общим секретом, метка времени внутри подписи, окно 5 минут — перехваченный запрос не повторить.
- **Ровно один раз.** Повтор события с тем же `event_id` принимается, но ничего не меняет — даже если пришёл одновременно с первым. Если реестр не ответил на наш запрос, событие не засчитывается, и его повторная доставка отработает полностью.
- **Ничего о чужих.** На событие об участнике, который у нас не привязан, ответ тот же — 202, и о нём ничего не сохраняется.
- **Демо.** Заглушка реестра (`fsp-mock`) умеет менять достижения участника через админку и отправляет такое событие в API — шаг описан в README.
- **С настоящим реестром** нужны адрес для событий и общий секрет (`FSP_REGISTRY_WEBHOOK_SECRET`); вместо статического токена чтения — client credentials realm `fsp` (§7.1). Ежедневная фоновая сверка — страховка от потерянных событий, следующий шаг.

8. Безопасность и 152-ФЗ

8.1. Аутентификация

- Пароли — Argon2id (19 МиБ, t = 2, p = 1); при неизвестном email выполняется та же проверка, чтобы время ответа не раскрывало наличие аккаунта.
- Короткий access-токен (JWT EdDSA, 15 минут) и ротируемый refresh-токен (30 дней) в httpOnly-cookie; повторное использование старого refresh-токена отзывает всю цепочку. Выход заносит access-токен в deny-list. Сам токен скрипту страницы недоступен; рядом лежит cookie только со сроком его действия, и, когда срок вышел, кабинет обновляет сессию до запроса, а не после отказа 401.
- Защита от CSRF: изменяющие запросы с cookie принимаются только с собственного Origin.
- Ограничение частоты: вход, регистрация, повторные письма, приглашения, жалобы.

8.2. Персональные данные

- Email, ФИО и контакты кандидата хранятся зашифрованными (XChaCha20-Poly1305); поиск по email — через keyed-хеш (blind index).
- Версионированные согласия на обработку и публикацию с фиксацией времени, IP и User-Agent. Каждый документ знает, кому он адресован: обработка и соглашение — обеим ролям, публикация и видеозапись — только кандидату.
- **Публикация только с согласием.** Показать профиль в поиске можно лишь при действующих согласиях на обработку и на публикацию; иначе сервер отвечает 409 `consent_required` и называет документ, который нужно принять, — интерфейс тут же предлагает его текст. Отзыв любого из них сразу скрывает профиль. Дать согласие снова можно в настройках: принимается только действующая редакция документа, повторная выдача ничего не дублирует.
- **Двойная проверка в поиске.** Пул кандидатов для подборок, поиска по банку, статистики категорий, анонимного профиля и приглашений сам проверяет действующее согласие на публикацию — флага «видим» недостаточно. Пул и карточки кандидатов кешируются в памяти, но отзыв согласия, скрытие профиля и любая другая правка, меняющая выдачу или карточку, сбрасывают кеш до ответа. Каждая показанная карточка — в подборке, на странице поиска, в профиле — перепроверяется по базе: публикация, статус поиска и флаги приватности читаются при каждом показе. Поэтому скрытый профиль, имя, данные ФСП или компании в опыте исчезают сразу, без пересборки и на любой копии сервера, даже если запись прошла в обход API.
- **Ничего личного в кешах.** Ответы с персональными данными и всё, что требует входа или API-ключа, помечены `Cache-Control: private, no-store`: их не сохранит ни браузер на диск, ни прокси, ни CDN. Публично кешируются только справочники и опубликованные вакансии (§12.4).
- **Право на доступ (ст. 14).** Кнопка «Скачать мои данные» отдаёт JSON-файл со всем, что хранится об аккаунте: профиль, согласия с редакциями и датами, тесты с итогами (без самих заданий и ответов), история категории, приглашения, отклики, журнал доступа к контактам, сводки видеозаписи без видео, уведомления и настройки писем. Работодатель получает свой аккаунт, данные компании и свои уведомления.
- **Журнал доступа к контактам**: кто, когда, на каком основании. Кандидат видит его в настройках как «Кто открывал ваши контакты» — компания, время, основание (принятое приглашение или собственный отклик), без сотрудника и его IP.
- **Журнал модерации**: кто из модераторов, когда, что сделал, с каким объектом и почему (комментарий обязателен). Пишется в одной транзакции с самим действием, не правится и не удаляется, переживает аккаунты. Модератор смотрит его в своём кабинете с фильтрами.
- **Просмотр видеозаписи модератором** — тоже строка журнала, записанная до того, как ушёл первый байт видео; проверка «есть ли файл» просмотром не считается. Кандидат видит дату просмотра у самого теста в истории («Запись теста просмотрена модератором: 6 октября, 14:35») и в выгрузке своих данных. Мы сознательно не смешиваем это с «Кто открывал ваши контакты»: запись — не контакты.
- **Минимум данных у модератора**: в карточке спорного теста нет ФИО, контактов и почты — только анонимный код; решение принимается по ответам, событиям и записи.
- **Передача в ATS компании** — только после принятия приглашения или отклика и тем же путём, что и в кабинете; каждая передача видна кандидату в том же журнале. Дальше компания обрабатывает данные в своей ATS как самостоятельный оператор (§10.2).
- **Политика обработки персональных данных** (ст. 18.1) — публичная страница `/privacy`: состав данных, цели, основания, кому видны, сроки, cookies, права и как ими пользоваться в кабинете, действующие тексты согласий.

- **Уведомления и письма.** В уведомлении — только то, что получатель и так видит в кабинете. В письмах компаниям нет ФИО и контактов кандидата — только анонимный код и ссылка в кабинет, где контакты открываются по обычным правилам; это заложено в типах: в данных уведомления для компании просто нет полей для имени и контактов, а свободный комментарий кандидата к отказу письмо не цитирует. Отписаться можно из любого письма без входа. Уведомления, настройки и список отправленных писем входят в выгрузку данных и удаляются вместе с аккаунтом; уведомления о кандидате или компании удаляются и тогда, когда удаляется их аккаунт. В уведомлениях о решениях модерации модератор не назван, а компания не узнаёт, кто на неё жаловался и что написал: только исход, свою вакансию по названию и число подтверждённых жалоб; жалобу на приглашение не выдаёт даже название приглашения.
- Удаление аккаунта стирает персональные данные, связи обезличиваются.
- Хранение — на сервере в РФ.

Соискатели 14–17 лет — наше дополнение: среди участников ФСП много школьников, поэтому правило «18+ или согласие законного представителя».

- **Возраст при регистрации — одним вопросом.** «Мне есть 18 лет» — и больше ничего: дату рождения у взрослых не спрашиваем, ответ сохраняется вместе с выдачей согласий. «Мне меньше 14» — доброе сообщение, что регистрация открывается с 14 лет; аккаунт не создаётся. «Мне 14–17 лет» — дата рождения (сервер проверяет, что это действительно 14–17), ФИО и email законного представителя. Компания при регистрации подтверждает, что её представителю есть 18.
- **Письмо представителю** уходит, только когда подросток подтвердил свою почту. В письме две ссылки: ответить (одноразовая, 7 дней) и отозвать согласие (работает, пока согласие действует). Обе ведут на публичную страницу `/guardian/confirm`: кто мы, какие данные, что компании видят анонимную карточку, контакты — только после того, как подросток сам примет приглашение, видео — только с отдельного согласия, как отозвать. Повторная отправка и смена адреса из кабинета выпускают новые ссылки и гасят старые; лимит — 3 письма в час на подростка и 5 в сутки на один адрес.
- **Согласие представителя** — отдельный версионированный документ `guardian_consent`: выдача записывается на подростка с ФИО и email представителя (зашифрованы, у email — blind index), временем, IP и браузером. Сам подросток выдать или отозвать его не может.
- **Публикация.** Пока согласия нет, кабинет и тест доступны, но профиль не показывается: `PUT /candidate/privacy` отвечает 409 `guardian_consent_required`, а пул поиска сам проверяет согласие представителя у всех, кому нет 18, — та же двойная проверка, что и для согласия на публикацию. Отклик на вакансию и принятие приглашения, которые открывают компании контакты, до согласия тоже недоступны. Отказ или отзыв сразу скрывает профиль, а баннер в кабинете объясняет почему и предлагает отправить письмо ещё раз или указать другой адрес.
- **18 лет — без расписаний.** «Несовершеннолетний» вычисляется из даты рождения в момент запроса, поэтому в день 18-летия ограничения снимаются сами.
- **Работодатель** видит нейтральную отметку «14–17 лет» (дату рождения — никогда), в окне приглашения — короткую памятку об ограничениях Трудового кодекса для работников до 18 лет (ст. 63, 92, 94, 265), в поиске по базе и в подборке — фильтр «Только 18+». Это фильтр, а не понижение в выдаче.
- **Срок хранения.** Через 30 дней после 18-летия ФИО, email, IP и браузер представителя и сама дата рождения стираются; остаётся факт: редакция согласия, время выдачи и отзыва. Очистка запускается при старте сервера и каждые 6 часов. При удалении аккаунта данные представителя стираются сразу. Выгрузка данных содержит возрастную группу, дату рождения подростка и записи о согласии представителя.

Видеозапись теста (§4.7):

- **Согласие** — отдельный версионированный документ «Согласие на видеозапись во время теста», при регистрации не запрашивается. Его выдача фиксируется так же, как остальные: время, IP, User-Agent. Без согласия запись не начинается: сервер отклоняет старт теста с камерой, если не подтверждена текущая версия документа.
- **Что именно:** только изображение с камеры, без звука и содержимого экрана; запись не используется для распознавания лица и установления личности.
- **Шифрование.** Каждый 10-секундный фрагмент шифруется XChaCha20-Poly1305 тем же модулем, что и персональные данные, и привязывается к своей сессии и номеру: файл, перенесённый в чужую запись или на другое место, не расшифруется. Файлы лежат в отдельном каталоге (в Docker — отдельный том), в базе — только метаданные: выбор камеры, ссылка на согласие, номера и размеры фрагментов, события. Запись отдаётся потоком и расшифровывается по одному фрагменту.

- **Срок хранения** — 90 дней. Фоновая задача удаляет просроченные записи при старте сервера и каждые 6 часов; сводка и отметка остаются.
- **Отзыв согласия** в настройках сразу удаляет все записи кандидата; **удаление аккаунта** удаляет их вместе с остальными данными.
- **Лимиты:** фрагмент до 1 МиБ, запись до 64 МиБ, не больше 50 событий за запрос и 500 на тест.

8.3. Разграничение доступа

- Роли кандидата, работодателя и модератора проверяются на уровне маршрутов; кандидат видит только свои приглашения, отклики и тесты, чужие ресурсы возвращают 404. Все методы модерации отвечают кандидату и работодателю 403 — это проверяет black-box тест, который обходит все операции модерации из опубликованной спецификации; роль модератора регистрацией не выдаётся.
- **Контакты закрыты типами.** Карточка кандидата для работодателя не содержит полей контактов. Контакты можно получить только из «основания доступа», которое выдаёт единственная функция: она проверяет принятое приглашение или отклик и пишет журнал.
- **Видео закрыто так же.** В карточке для работодателя есть только отметка о видеозаписи — статус и число замечаний, без идентификатора сессии и ссылок. Запись отдают два метода: кандидату — его собственная, администратору — любая, с записью в журнал. Для работодателя оба отвечают 403: анонимность сохраняется до принятия приглашения, а видео не раскрывается и после.

8.4. ГОСТ и сертифицированная криптография

Алгоритм хранится вместе с данными (строка хеша PHC, версия шифротекста), хеширование и шифрование — за интерфейсами. Переход на ГОСТ (PBKDF2-HMAC-Стрибог по Р 50.1.111-2016, «Кузнечик») и сертифицированное СКЗИ — это новая реализация интерфейса без изменения схемы данных; в MVP ГОСТ не используется, так как соответствие обеспечивает сертификация СКЗИ, а не сам алгоритм.

9. API

Все методы описаны в OpenAPI 3.1: спецификация `/api/v1/openapi.json`, Swagger UI — `/api/docs`. 130 операций на 110 путях; у основных запросов — примеры тел, у созданных ресурсов — заголовок `Location`.

- Растущие списки (приглашения, отклики, ответы на задания, журнал доставок, очереди модерации, уведомления) — страницами по непрозрачному курсору `cursor` / `next_cursor`. Повтор безопасен: вакансия и приглашение с одним `Idempotency-Key` создаются один раз, импорт вакансии из ATS с тем же `external_id` возвращает уже созданную потребность, повторная жалоба на тот же объект — ту же открытую жалобу.
- Версионирование: префикс `/api/v1`; несовместимые изменения — новая версия рядом.
- Формат: JSON, даты RFC 3339, деньги — целые рубли.
- Ошибки — RFC 9457 `application/problem+json` со стабильным машинным полем `code` (например, `grade_change_cooldown`, `invitation_quota_exceeded`, `contacts_not_granted`, `consent_required` — с полем `document_id` документа, который нужно принять); 404 и для чужих ресурсов; 409 — конфликт состояния; 422 — ошибки полей; 429 — лимит с `Retry-After` — сколько секунд до сброса окна. Так же отвечает и запрос, который не удалось разобрать: `invalid_json`, `unsupported_media_type`, `invalid_path`, `invalid_query`, `payload_too_large`; в OpenAPI эти ответы указаны у каждого метода, где они возможны.
- Аутентификация: httpOnly-cookie или заголовок `Authorization: Bearer`. API для ATS работодателя — только API-ключ компании `Bearer fsp_live_…` со скоупами.
- Служебные пути вне спецификации: `/healthz` (живость), `/readyz` (база и Valkey), `/metrics` (Prometheus, наружу не отдаётся). Соглашения и полный список с ролями и кодами ошибок — `docs/api.md` в репозитории.

Группа	Что делает	Операций
`auth`	Регистрация с согласиями, подтверждение email, вход, обновление и завершение сессии	6
`account`	Текущий пользователь, согласия: выдача, отзыв, повторная выдача; выгрузка всех данных, удаление аккаунта, жалобы	7
`catalog`	Справочники, автодополнение навыков, документы согласий	3
`candidate`	Профиль, PDF-резюме и PDF-профиль, приватность, журнал доступа к контактам, опрос, статистика спроса; для 14–17 лет — статус согласия представителя, повторное письмо, смена адреса	13
`assessment`	Старт теста, текущее задание, ответ, история, категория, план теста под вакансию	8
`proctoring`	Шаг перед тестом, фрагменты видеозаписи, журнал событий, просмотр своей записи	4
`fsp`	Привязка ФСП ID (OIDC), dev-привязка, достижения, синхронизация, отвязка; приём подписанных событий реестра	7
`employer`	Профиль компании	2
`matching`	Потребности, распознавание текста вакансии, категории, подборка, поиск по банку, анонимный профиль	13
`invitations`	Приглашения с обеих сторон, квота и статистика, контакты	10
`vacancies`	Вакансии, публичный список, отклики	11
`tasks`	Регулярные задания	8
`guardian`	Публичная страница законного представителя по ссылке из письма: что он подтверждает; ответ — подтвердить, отказаться, отозвать	2
Валидация	Проверка подбора на своём наборе вакансий, резюме и разметки (§6.4)	1
`trust`	Уровень доверия и квота компании с причинами и решениями модератора, проверка вилки по рынку категории, поиск похожих собственных вакансий	3
`Модерация`	Кабинет модератора: счётчики, очередь тестов, карточка теста, запись (каждый просмотр — в журнал), решение; очередь антифейка и жалоб, действия; журнал. Списки — с курсором	9
`integrations`	Кабинет «Интеграции»: вебхук, смена секрета, тестовое событие, журнал доставок и повтор, API-ключи	10
`Интеграции ATS`	API для ATS по ключу: опрос приглашений, профиль и PDF после согласия, этапы найма, импорт вакансий	6
`Уведомления`	Лента уведомлений с пагинацией, отметка прочитанного, счётчик непрочитанных, настройки писем по видам, отписка в один клик по ссылке из письма	7

10. Антифейк и концепции вне MVP

10.1. Защита от фиктивных вакансий и недобросовестных работодателей

ТЗ относит антифейк к концепциям вне MVP. У нас работает его основная часть. Главный принцип — **ни один сигнал не блокирует публикацию или приглашение**: эвристика ошибается, и честная компания, которая платит выше рынка, не должна от этого страдать. Сигнал делает три вещи: мягко предупреждает работодателя в форме, учитывается в уровне доверия к компании (и через него в квоте приглашений) и попадает в очередь модерации.

Базовые ограничения работали и раньше: вилка «от–до» обязательна, не больше одного действующего приглашения на пару «компания–кандидат», приглашение живёт 14 дней, кандидат может пожаловаться на вакансию, приглашение или компанию.

Сигнал	Правило	Работодатель видит	Кандидат видит
Корпоративный домен	Подтверждённая почта для входа и сайт компании на одном домене; поддомены подходят в обе стороны (`hr@corp.ru` и `jobs.corp.ru`). Публичные почтовые сервисы не засчитываются никогда: gmail.com, yandex.ru, ya.ru, mail.ru, bk.ru, inbox.ru, list.ru, rambler.ru, outlook.com, icloud.com, proton.me и другие, включая их поддомены. Общие зоны (`com.ru`, `msk.ru`, `github.io`, `tilda.ws` и т. п.) и голые TLD не считаются доменом компании. Домены сравниваются в ASCII: `пример.рф` и `xn--e1afmkfd.xn--p1ai` — один сайт	Статус и подсказку, как получить отметку: какой домен у почты, какой у сайта, что поправить	Нейтральную отметку «Домен подтверждён» у вакансии и приглашения. Если отметки нет, не показываем ничего
Вилка выше рынка	Верх вилки больше 1,5 × 90-го перцентиля ожиданий кандидатов категории — тех же, что в карточках категорий: опубликованных, а в 14–17 лет — только с согласием законного представителя. Предложения обычно на 10–30 % выше ожиданий, поэтому щедрая, но реальная вилка под порог не попадает. Если в категории меньше 5 указанных ожиданий, проверка не выполняется	Предупреждение в форме вакансии и приглашения, рядом медиана и 90-й перцентиль рынка категории	Ничего
Слишком широкая вилка	Верх ⩾ 3 × низ: это разброс от джуна до синьора, а не одна позиция (классика фейка — «от 50 до 500 тысяч»)	То же	Ничего
Дубль своей вакансии	Пары соседних слов текста (название + описание) сжимаются в MinHash-подпись из 128 хешей; доля совпавших хешей оценивает долю общих пар слов. Порог — 0,6: правки и перестановка абзацев оставляют сходство выше 0,75, две разные вакансии одной компании с общим блоком «о компании» дают около 0,2	Перед сохранением: «Похожая вакансия уже есть» со ссылкой — предложение обновить её; сохранить всё равно можно	—
Копия чужой вакансии	То же сходство с опубликованной за 90 дней вакансией другой компании	В уровне доверия: «вакансия почти дословно совпадает с вакансией другой компании»	—
Пересказ чужой вакансии	Эмбеддинги (§5.6): косинус текста с опубликованной за 90 дней вакансией другой компании ⩾ 0,953. Порог — без ложных срабатываний на калибровочном наборе: 12 демо-вакансий, их пересказы, разные вакансии и пять похожих «не копий» с той же ролью и стеком (`docs/validation/paraphrase.md`): ловит половину пересказов, MinHash — ни одного. Проверка идёт через несколько секунд после публикации, когда посчитан вектор. Квоту не снижает: сходство по смыслу ошибается чаще дословного — решает модератор	В карточке вакансии: «По смыслу описание очень похоже на вакансию другой компании. Если это совпадение — ничего делать не нужно: квоту это не снижает, вакансию посмотрит модератор» — без названия чужой компании	—
Поведение	Жалобы кандидатов (уникальные авторы, не отклонённые модератором, за 90 дней); доля отказов с причиной «зарплата» или «компания»; доля отозванных приглашений; доля приглашений, оставшихся без ответа	В уровне доверия с цифрами	—

Почему кандидат не видит подсказку о вилке. Высокая вилка сама по себе не улика: компания может честно платить больше рынка, а наша статистика — ожидания кандидатов, а не реальные предложения. Нейтральная фраза «Вилка выше обычной для этой категории» на практике читается либо как тревога и бьёт по честной компании, либо как приманка и помогает фейку. Поэтому сигнал показываем там, где с ним можно что-то сделать: работодатель исправит опечатку прямо в форме, модератор увидит вакансию в очереди, а квота снизится, если такие вилки повторяются.

Уровень доверия и квота приглашений. Правило одинаковое для всех и видно работодателю целиком:

- новая компания начинает с 10 приглашений в неделю;
- +10 за подтверждённый домен, +10 после 30 дней на платформе, +10, когда кандидаты примут 3 приглашения (+5 — после первого принятого). Бонусы складываются до +20: любых двух шагов хватает до максимума 30, так что компания с публичной почтой тоже может его достичь;
- −10 за каждую жалобу кандидата за 90 дней (не больше −30); −10, если у 40 % и больше отказов (при 5+ отказах) причина «зарплата не соответствует» или «компания»; −5 за каждую вакансию или приглашение с вилкой выше рынка или слишком широкой (не больше −10); −20, если есть вакансия, скопированная у другой компании;
- −5, если отозвано 30 % и больше из 5+ приглашений; −5, если кандидаты ответили на 10 % и меньше из 10+ закрытых приглашений; −5, если есть копии собственных вакансий;
- квота не опускается ниже 5 — одна ошибка или одна несправедливая жалоба не останавливает работу. Отметки на вакансиях перестают учитываться через 90 дней.

Уровень — для подписи: «Новая компания» (меньше месяца, замечаний нет), «Обычный», «Высокий» (квота 30) и «Квота временно снижена» (есть замечания и квота опустилась до 10 и ниже). В кабинете работодателя раздел «Доверие и квота» показывает текущую квоту как сумму «база + бонусы − замечания», каждую причину с её вкладом — так же, как объяснение балла в подборке, — и конкретные шаги, которые её поднимут. Формулировки без обвинений: «Есть замечания — они перестают учитываться через 90 дней».

Модерация. Сигналы ничего не блокируют — блокирует только решение человека. В кабинете модератора одна очередь «Антифейк и жалобы» сводит жалобы кандидатов, вакансии и приглашения с отметками и компании с отрицательными сигналами, сгруппированные по компаниям: первой идёт компания, чей сигнал ждёт дольше всех; у каждого элемента — причины словами («Верх вилки 800 000 ₽ — в 1,9 раза выше 90-го перцентиля ожиданий Бэкенд-разработка · Middle (420 000 ₽)»), уровень доверия и квота компании, текст жалобы без автора. Действия — каждое с комментарием, который компания увидит в разделе «Доверие и квота»:

Действие	Что происходит
Снять: нарушений нет	Отметка перестаёт снижать квоту; жалоба закрывается как необоснованная и тоже перестаёт её снижать; компания уходит из очереди до следующей жалобы или отметки
Снять вакансию с публикации	Вакансия закрывается и пропадает из публичного списка, компания не может опубликовать её снова; жалобы на неё закрываются как подтверждённые и продолжают считаться свои 90 дней
Приостановить приглашения на N дней (1–90)	Квота приглашений — 0 до конца срока, ниже обычного минимума 5: это решение человека, а не эвристика. Жалобы на компанию и её приглашения закрываются как подтверждённые. Работодатель видит баннер с датой, отправка приглашения отвечает понятной ошибкой
Вернуть	Снятая вакансия снова публикуется, приостановка снимается досрочно

Компания видит каждое решение о себе, своих вакансиях, приглашениях и жалобах на них в разделе «Доверие и квота» — что произошло и комментарий модератора; кто пожаловался, не раскрывается. От одного человека на один объект открыта одна жалоба: повторная отправка возвращает ту же и не снижает квоту ещё раз. Все действия попадают в журнал модерации; журнал ищется по анонимному коду кандидата. Отметки хранятся вместе с вакансией и приглашением (JSON с видом сигнала и цифрами на момент публикации), подпись текста — 512 байт в строке вакансии, поэтому новая вакансия сравнивается с тысячами недавних за доли миллисекунды без отдельного индекса.

Логика — чистый модуль `matching::trust` без ввода-вывода и часов; пороги и правило квоты покрыты модульными тестами, как и выбор отметок «дубль своей вакансии» и «копия чужой»; предупреждение о вилке в форме, поиск похожей вакансии до сохранения и уровень доверия новой компании проверяют black-box тесты (§6.6).

Следующие шаги.

- Проверка ИНН и ОГРН по ЕГРЮЛ (открытый сервис ФНС): название, статус «действующее», дата регистрации; несовпадение названия или ликвидированное юрлицо — сильный отрицательный сигнал.
- Подтверждение домена DNS-записью TXT — для компаний, которые не могут войти с корпоративной почты.
- Отчётность: доля отмеченных вакансий, время реакции модератора, жалобы на 100 приглашений — по неделям.
- Порог пересказа по разметке модераторов: сейчас он выбран по 12 пересказам и 76 отрицательным парам.

10.2. Интеграция с ATS работодателя

ТЗ относит интеграцию с внутренними ATS к концепциям. Мы сделали её рабочим универсальным слоем, через который подключается любая система подбора, умеющая принять HTTPS-запрос или опрашивать REST API. Нативные коннекторы к конкретным ATS — тонкие адаптеры поверх этого слоя, их план — в конце раздела. В демо-стенде компания `employer@demo.fsp` уже подключена к демо-ATS (`http://localhost:8082`): события о приглашениях приходят туда с подписью, а кнопки «Собеседование», «Оффер», «Нанят», «Отказ» возвращают этап кандидату.

Как устроено

```
кандидат принял приглашение
        │  одна транзакция PostgreSQL
        ▼
invitations.status = accepted
  + invitation_events + contact_access_log (канал webhook)
  + integration_events: invitation.accepted (тело зашифровано)
        │  диспетчер в каждой реплике API: FOR UPDATE SKIP LOCKED,
        │  аренда 60 с, подпись HMAC, DNS только в публичные адреса
        ▼
ATS работодателя ── API-ключ fsp_live_…, скоупы ──► /api/v1/integrations
    GET  /invitations       опрос вместо вебхука
    GET  /candidates/{id}   профиль и PDF после согласия (ContactGrant, журнал)
    POST /outcomes          собеседование, оффер, найм, отказ → кандидату
    POST /needs             текст вакансии → потребность и подборка
```

Часть	Что делает
Outbox событий	Событие записывается в той же транзакции, что и смена статуса. Если запись не удалась, откатываются и статус, и запись в журнале доступа — состояние платформы и поток событий не расходятся
Диспетчер	Фоновая задача в каждой реплике API. Берёт до 16 готовых событий с `FOR UPDATE SKIP LOCKED` и сразу сдвигает время следующей попытки на 60 секунд: реплики не отправят одно событие дважды одновременно, а упавшая реплика его не потеряет. Повторы через 1 мин, 5 мин, 30 мин, 2 ч и 12 ч, пока очередной укладывается в сутки от создания; дальше событие помечается недоставленным (у адреса, который не отвечает, — после шестой попытки, примерно через 14,6 ч)
Вебхук компании	Один HTTPS-адрес на компанию. Секрет подписи показывается один раз; его можно сменить (сутки события подписаны обоими секретами) или отключить вебхук целиком
Журнал доставок	Каждая попытка: код ответа, задержка, ошибка. Кнопки «Отправить ещё раз» и «Тестовое событие» — в кабинете «Интеграции»
API-ключи	`Authorization: Bearer fsp_live_…`, скоупы `invitations:read`, `candidates:read`, `outcomes:write`, `needs:write`. Ключ работает только в дереве `/api/v1/integrations`
Этапы найма	ATS отмечает собеседование, оффер, найм или отказ. Кандидат видит «Компания отметила: собеседование» в приглашении или отклике, работодатель — этапы в воронке на главной
Импорт вакансий	Текст вакансии из ATS разбирается тем же разборщиком, что и форма потребности: специализация, грейды, стек, вилка, формат. В ответе — id потребности и ссылка на подборку. С `external_id` (id вакансии в ATS) повтор ничего не создаёт и возвращает ту же потребность
Опрос вместо вебхука	`GET /integrations/invitations` отдаёт изменения приглашений по возрастанию с курсором «время + id». Изменения последних секунд и транзакций, которые ещё пишут, приходят следующим опросом: изменение, сохранённое раньше, но закоммиченное после опроса, не перепрыгивается курсором

Каталог событий. Все события приходят одним конвертом `{id, type, created_at, company_id, data}`; `id` совпадает с заголовком `FSP-Event-Id`. Доставка гарантирована «хотя бы один раз», поэтому ATS отбрасывает повторы по этому id.

Событие	Когда	Что о кандидате
`invitation.sent`	компания отправила приглашение	анонимный код и категория
`invitation.viewed`	кандидат открыл приглашение	анонимный код и категория
`invitation.accepted`	кандидат принял	id, ФИО, телефон, Telegram, email, ссылки на профиль и PDF
`invitation.declined`	кандидат отклонил	анонимный код и категория; причина и комментарий
`invitation.expired`	прошло 14 дней без ответа	анонимный код и категория
`invitation.withdrawn`	компания отозвала	анонимный код и категория
`application.created`	кандидат откликнулся на вакансию	id, ФИО и контакты: кандидат откликнулся сам
`application.status_changed`	отклик сменил статус	id, ФИО и контакты; после отзыва отклика — только код
`contacts.opened`	сотрудник открыл контакты в кабинете	id, основание и время, без самих контактов
`integration.test`	кнопка «Тестовое событие»	—

До принятия:

```
{
  "id": "0199b1c2-7f3e-7c3a-9d1e-2f6a8b0c4d11",
  "type": "invitation.sent",
  "created_at": "2026-10-06T09:12:44Z",
  "company_id": "0199a0f1-…",
  "data": {
    "invitation": {
      "id": "0199b1c2-…", "status": "sent",
      "title": "Backend-разработчик в платёжную команду",
      "salary": { "min": 280000, "max": 340000 }, "work_format": "remote",
      "expires_at": "2026-10-20T09:12:44Z", "updated_at": "2026-10-06T09:12:44Z"
    },
    "candidate": {
      "code": "K-7F3A2C",
      "category": { "specialization": "backend", "grade": "middle" }
    }
  }
}
```

После принятия у того же приглашения:

```
"candidate": {
  "code": "K-7F3A2C",
  "category": { "specialization": "backend", "grade": "middle" },
  "id": "0199a3d0-…",
  "disclosed": {
    "full_name": "Анна Смирнова", "phone": "+7 900 123-45-67",
    "telegram": "@anna_dev", "email": "anna@example.ru", "basis": "invitation",
    "profile_url": "https://…/api/v1/integrations/candidates/0199a3d0-…",
    "profile_pdf_url": "https://…/api/v1/integrations/candidates/0199a3d0-…/profile.pdf"
  }
}
```

Этап найма из ATS обратно:

```
POST /api/v1/integrations/outcomes
Authorization: Bearer fsp_live_…

{ "invitation_id": "0199b1c2-…", "stage": "interview",
  "reason": "Зовём на техническое интервью", "at": "2026-10-08T15:00:00+03:00" }
```

Повтор того же этапа обновляет причину и время, а не создаёт дубль — ATS может безопасно повторять запрос. Этап по непринятому приглашению отклоняется (409): до согласия кандидата компании нечего отмечать.

Безопасность

- **Подпись.** Заголовок `FSP-Signature: t=<unix>,v1=<hex>`, где `v1 = HMAC-SHA256(секрет, t + "." + тело)`. Метка времени входит в подпись, а получатель отклоняет запросы старше 5 минут, поэтому перехваченное событие нельзя отправить повторно позже. Секрет хранится зашифрованным и привязан к компании.
- **Защита от SSRF.** Адрес компании — это ввод пользователя, и платформа не должна стать прокси во внутреннюю сеть. Принимаются только `https://` без логина в адресе, редиректы не выполняются. Адрес проверяется внутри DNS-резолвера HTTP-клиента при каждом соединении: непубличные адреса имени (loopback, частные сети, link-local с 169.254.169.254, CGNAT, служебные диапазоны, включая IPv6 и IPv4 внутри IPv6) отбрасываются, соединение идёт только на публичные, а имя без публичных адресов отклоняется. Поэтому имя, которое после проверки начали резолвить в 127.0.0.1 (DNS rebinding), во внутреннюю сеть не попадёт. Исключение — хосты из `WEBHOOK_DEV_HOSTS`: в демо так подключён контейнер демо-ATS.
- **Ключи.** `fsp_live_` и 256 случайных бит. В базе только SHA-256 и видимый префикс: при такой энтропии медленный хеш не нужен, а по утёкшей базе ключ не восстановить. Ключ показывается один раз, отзывается мгновенно, у каждого видно время последнего запроса. Лимит — 120 запросов в минуту на ключ.
- **Скоупы и границы.** Ключ без `candidates:read` не получит профиль, без `outcomes:write` не отметит этап. Дерево `/api/v1/integrations` принимает только ключ — cookie и JWT туда не доходят, а с ключом не открыть ни один метод кабинета.

152-ФЗ

- **Что и когда уходит в ATS.** До принятия приглашения — только анонимный код, категория и данные самого приглашения: должность, вилка, формат, статус, причина отказа. ФИО, контакты и PDF-профиль попадают в событие только после того, как кандидат принял приглашение или сам откликнулся. Это то же основание, по которому компания видит контакты в кабинете, и в коде путь тот же: данные раскрываются только через `ContactGrant`. Видео теста, ответы на задания и данные других компаний не уходят никогда.
- **Каждая передача в журнале.** Кандидат видит в «Кто открывал ваши контакты» и открытие в кабинете, и передачу в ATS компании — с пометкой «передано в систему подбора компании».
- **Кто отвечает дальше.** Получив данные после согласия кандидата, компания обрабатывает их в своей ATS как самостоятельный оператор — для своего подбора, по своим правилам хранения. Об этом сказано в политике обработки `/privacy`. Отключение вебхука или отзыв ключа прекращает передачу; удалить уже переданное кандидат может, обратившись к компании.
- **Минимизация у нас.** Тело события в очереди зашифровано тем же XChaCha20-Poly1305, что и поля ПДн, с привязкой к id события. Завершённые события удаляются через 30 дней, а при удалении аккаунта кандидата его события удаляются сразу. Компания без вебхука событий не получает вовсе, значит и раскрытия нет.

Этапы найма как разметка. Главный пробел валидации подбора — нет экспертной разметки, кто из подборки на самом деле подошёл (§6.4). Этапы из ATS — это настоящие исходы найма по конкретным приглашениям: дошёл до собеседования, получил оффер, вышел на работу. Они хранятся вместе с приглашением, а оно связано с потребностью и подборкой, где у кандидата был свой балл. Накопив исходы, можно калибровать веса ранжирования на реальных данных, а не на синтетике: подбирать веса признаков так, чтобы нанятые оказывались выше в подборке.

Нативные коннекторы. Универсальный слой уже покрывает ATS, которые принимают вебхуки или умеют опрашивать REST. Нативный коннектор — отдельный воркер, который читает тот же outbox как внутренний вебхук и переводит события в API конкретной системы. Его настройки в кабинете: токен ATS (хранится зашифрованным), вакансия по умолчанию и таблица соответствия статусов компании нашим этапам. Сведения о Huntflow, «Потоке» и Talantix взяты из их опубликованной документации API; про E-Staff и Skillaz открытой спецификации мы не нашли, для них описан подход.

Система	Авторизация	Мы → ATS (после согласия кандидата)	ATS → мы (этапы найма)
Huntflow	Bearer-токен из настроек Huntflow: `access_token` живёт 7 дней, `refresh_token` — 14, обновление `POST /v2/token/refresh`	`POST /accounts/{account_id}/applicants` (ФИО, телефон, email, соцсети), `POST /accounts/{account_id}/upload` (PDF), `POST /accounts/{account_id}/applicants/{applicant_id}/vacancy` (на вакансию, со статусом и файлами)	вебхук Huntflow `POST /accounts/{account_id}/hooks` с событиями `APPLICANT`; статусы — `GET /accounts/{account_id}/vacancies/statuses`
Поток	Токен из раздела «Генерация токенов»: `Authorization: Bearer …` или параметр `token`; база `https://app.potok.io/api/v3`; лимит 100 запросов за 10 с	`POST /applicants.json` (кандидат), `POST /applicants/{applicant_id}/resume_file.json` (PDF), `POST /ajs_joins.json` (на вакансию)	исходящих вебхуков в публичной спецификации v3 нет — опрос истории `GET /jobs/{job_id}/events.json` и карточки `GET /applicants/{applicant_id}.json`
Talantix (hh.ru)	Токен из личного кабинета Talantix; GraphQL API	мутация `createPerson`, перевод на этап воронки `transferPersonsToWorkflowStatus`	исходящих уведомлений в документации мы не нашли — периодический опрос GraphQL API
E-Staff	По договорённости с вендором; часто стоит внутри периметра компании	ATS сама забирает данные: `GET /integrations/invitations` и `GET /integrations/candidates/{id}` — входящие соединения из интернета не нужны	`POST /integrations/outcomes` с ключом
Skillaz	По договорённости с вендором	вебхук на адрес интеграции Skillaz или опрос нашего API	`POST /integrations/outcomes` с ключом

Соответствие полей и статусов — одно для всех коннекторов:

У нас	Huntflow	Поток	Talantix
ФИО, телефон, email, Telegram	`last_name`, `first_name`, `middle_name`, `phone`, `email`, `social`	карточка кандидата `POST /applicants.json`	персона (`createPerson`)
PDF-профиль	файл через `upload`, затем `files` при добавлении на вакансию	файл резюме `resume_file.json`	ссылка на PDF в комментарии к персоне (`createPersonComment`)
Потребность / вакансия	вакансия `POST /accounts/{account_id}/vacancies`	вакансия `POST /jobs.json`, потребность `POST /demands.json`	заявка на подбор `createHiringRequest`
`interview`, `offer`	пользовательские статусы компании — по таблице соответствия в кабинете	этап воронки (`stage_type`) — по таблице	этап воронки (`workflowStatus`) — по таблице
`hired`	статус `hired` (с квотой и датой выхода)	статус трудоустройства финалиста	найм персоны (`hirePersons`)
`rejected`	статус `rejected` с причиной отказа (`rejection_reasons`)	отклонение `POST /jobs/{job_id}/{applicant_id}/decline.json`	отказ персоне (`discardPersons`)

Источники: (спецификация `https://api.huntflow.ru/v2/openapi.json`), и , и .

10.3. Масштабирование

API без состояния за балансировщиком; общие лимиты и поколение кешей в Valkey; реплика PostgreSQL на чтение; эмбеддер — отдельный сервис, его можно масштабировать независимо, а воркер очереди эмбеддингов есть в каждой реплике API и делит очередь через `FOR UPDATE SKIP LOCKED`; аналитика событий — выгрузка изменений в ClickHouse.

11. Библиотеки и компоненты

Весь код и зависимости — с открытыми лицензиями (MIT, Apache-2.0, BSD, ISC, MPL, OFL для шрифтов); веса модели эмбеддингов — MIT. Версии — из `Cargo.lock` и `web/package-lock.json`. Исключение — GSAP: распространяется бесплатно, в том числе для коммерческого использования, по собственной Standard License, которая не входит в число лицензий, одобренных OSI. GSAP отвечает только за анимации публичного сайта: без него страницы работают, а при `prefers-reduced-motion` анимации отключаются.

Компонент	Версия	Назначение
Rust	1.85+ (проверено на 1.99)	Язык backend
axum / tokio / tower-http / axum-extra	0.8.9 / 1.53 / 0.6 / 0.12	HTTP-сервер, асинхронный рантайм, middleware, cookie
sqlx / pgvector (крейт)	0.8.6 / 0.4.2	PostgreSQL: пул, запросы с проверкой при компиляции, миграции; тип `vector` для sqlx
candle (core, nn, transformers) / tokenizers / safetensors	0.11.0 / 0.23.2 / 0.8.0	Эмбеддинги: BERT-энкодер на CPU без C-зависимостей, токенизатор Hugging Face (без onig и esaxx), чтение весов
intfloat/multilingual-e5-small (веса, MIT)	ревизия 614241f	Модель эмбеддингов, 384 измерения, RU + EN; сравнивалась с cointegrated/rubert-tiny2 (MIT)
utoipa / utoipa-axum / utoipa-swagger-ui	6.0 / 0.3 / 10.0	OpenAPI и Swagger UI
argon2 / chacha20poly1305 / ed25519-dalek / jsonwebtoken / hmac / sha2	0.5.3 / 0.10.1 / 2.2 / 11.1 / 0.12 / 0.10	Криптография, токены, подписи вебхуков
redis (для Valkey)	1.2	Лимиты, deny-list, поколение кеша кандидатов
lettre	0.11	Почта
reqwest (rustls)	0.13	Запросы к ФСП ID и реестру, вебхуки в ATS
arc-swap	1.9	Снимки пула кандидатов в API и кольцевой буфер событий демо-ATS — без блокировок
krilla / pdf-extract	0.8.2 / 0.12.1	Генерация и разбор PDF
metrics-exporter-prometheus / tracing	0.18 / 0.1	Метрики и логи
regex / rustc-hash / rand / rand_chacha	1.13 / 2.1 / 0.9 / 0.9	Разбор текста, словари, детерминированная генерация
Next.js / React / TypeScript	16.3 / 19.2 / 5.9	Frontend
TanStack Query / openapi-typescript	5.104 / 7.13	Данные и типы клиента из OpenAPI
Tailwind CSS / react-hook-form / zod / lucide-react	4.3 / 7.89 / 4.6 / 1.52	Стили, формы, иконки
GSAP (ScrollTrigger, SplitText, DrawSVG, ScrambleText, Draggable, Inertia, Flip, MorphSVG, MotionPath) / `@gsap/react` / Lenis	3.15 / 2.1 / 1.3	Анимации и плавный скролл публичного сайта
PostgreSQL (+pgvector) / Valkey	17 (pgvector 0.8) / 8	Хранилище, векторы и HNSW-индекс, кэш
Keycloak	26.7	ФСП ID (OIDC)
Caddy / Mailpit	2.11 / 1.31	Входной прокси с TLS, почта для демо
Prometheus / Loki / Alloy / Grafana	3.5 / 3.7 / 1.20 / 12.4	Наблюдаемость
JetBrains Mono	2.304	Фирменный шрифт ФСП: подписи и цифры интерфейса, PDF-профиль
Onest	Google Fonts через `next/font`	Шрифт текста и заголовков сайта

12. Сборка, развёртывание и запуск

12.1. Docker (рекомендуется)

Требования: Docker с Compose v2, 4 ГБ свободной памяти.

```
git clone https://github.com/K1NGSM4N3/fsp-hack && cd fsp-hack
cp .env.example .env
docker compose -f deploy/compose.yml --profile full up -d --build   # или make up
```

Что	Адрес
Приложение	http://localhost:8080
Swagger UI	http://localhost:8080/api/docs
Письма (подтверждение email, письма представителю)	http://localhost:8025
ФСП ID (Keycloak), админка — `/admin`	http://localhost:8081
Демо-ATS работодателя	http://localhost:8082
Grafana (дашборд API)	http://localhost:3001

Профили compose: ядро (Caddy, web, API, эмбеддер, PostgreSQL, Valkey, Mailpit) поднимается всегда, `fsp` добавляет Keycloak и заглушку реестра ФСП, `ats` — демо-ATS, `obs` — Prometheus, Loki, Alloy и Grafana, `full` — всё сразу. Наружу открыты только порты из таблицы; API, web, эмбеддер, база и Valkey доступны лишь внутри сети compose, `/metrics` через Caddy закрыт. Образ эмбеддера при сборке скачивает веса модели с Hugging Face по закреплённой ревизии и проверяет sha256 (~490 МБ); векторы демо-данных считаются за минуту-две после первого старта, до этого подбор работает на ключевых словах.

При первом старте API применяет миграции и заполняет демо-данные (`SEED_DEMO=true`, только на пустой базе): 6 вымышленных компаний, 12 вакансий, 240 кандидатов с симулированными тестами и описаниями задач на местах работы (по ним эмбеддинги сравнивают профиль с потребностью), у 15 привязан ФСП ID (у 13 из них есть достижения, двое — без истории ФСП); у большинства тест пройден с видеозаписью (самих видеофайлов в демо нет). Шестеро кандидатов — 14–17 лет: у четырёх согласие представителя подтверждено, двое ждут ответа. У демо-работодателя шесть потребностей — по одной на специализацию, и он уже подключён к демо-ATS. Пять компаний из шести входят с почты на домене своего сайта, поэтому у их вакансий есть отметка «Домен подтверждён»; у демо-работодателя её нет, и раздел «Доверие и квота» показывает, как её получить. Очереди модератора не пусты: семь тестов ждут разбора — три с нетипичной картиной ответов и уходами со вкладки, два с той же картиной без видеозаписи, два с переключениями окна при обычной картине; у «Логистики Плюс Цифры» две вакансии с отметками (широкая вилка и вилка выше рынка) и три открытые жалобы кандидатов.

Демо-доступы (пароль `demo12345`): работодатель `employer@demo.fsp`; работодатели с подтверждённым доменом `hr@pixel-manufactura.example`, `hr@oblakostroy.example`, `hr@medsignal.example`, `it-hr@logplus.example`, `career@shield-key.example`; кандидаты `candidate@demo.fsp`, `cand1@demo.fsp` … `cand239@demo.fsp`; кандидат 14–17 лет, ожидающий согласия представителя, — `minor@demo.fsp` (он занимает один из номеров `cand…`); кандидат, чей тест ждёт решения модератора, — `review@demo.fsp` (тоже один из номеров); модератор — `moderator@demo.fsp` (кабинет `/admin`; регистрацией эта роль не выдаётся); участники ФСП в Keycloak — `fsp-000001` … `fsp-000010`.

12.2. Локальная разработка без Docker

Требования: Rust ≥ 1.85, Node.js ≥ 20.9, PostgreSQL ≥ 14. Либо `nix develop` — окружение со всеми инструментами.

```
make db                 # база fsp_hack
SEED_DEMO=true make api  # API на :8000: миграции при старте, демо-данные; ФСП ID — dev-привязкой
make web                # интерфейс на :3000, /api/* проксируется на :8000
make embedder           # эмбеддер на :8093 (веса — make model, ~490 МБ, sha256 проверяется)
make validate           # отчёты валидации в docs/validation/ (модель эмбеддингов — в процессе)
make test               # модульные тесты, без базы и сети
make blackbox           # black-box тесты на свежем стеке в Docker (§6.6)
make docs               # этот документ в PDF и DOCX
```

Без Valkey лимиты и отозванные токены хранятся в памяти процесса, без SMTP ссылки из писем печатаются в лог API, без эмбеддера (`EMBEDDINGS=off`) близость описаний считается по ключевым словам. PostgreSQL нужен с расширением pgvector (оно есть в `nix develop`). Сборка не требует базы: запросы sqlx проверяются по метаданным из `.sqlx/`. После изменения SQL или миграций — `make db-prepare`.

12.3. Для VPS

- В `.env` — публичные адреса (`PUBLIC_URL`, `FSP_OIDC_ISSUER`, `FSP_OIDC_REDIRECT_URL`) и `COOKIE_SECURE=true`. Камера в браузере доступна только по HTTPS или на localhost; видеозапись отключается `PROCTORING=off`, файлы лежат в томе `proctoring-data`.
- Новые ключи `JWT_ED25519_SEED_B64`, `FIELD_ENC_KEY`, `BIDX_KEY` — по 32 случайных байта в base64 (`openssl rand -base64 32`), новые пароли PostgreSQL и Keycloak (`KC_BOOTSTRAP_ADMIN_*`), секрет клиента ФСП ID.
- Без демо: `SEED_DEMO=false`, пустые `WEBHOOK_DEV_HOSTS` и `ATS_DEMO_*` — тогда вебхуки уходят только по HTTPS на публичные адреса.
- Эмбеддер: до ~250 МБ памяти под нагрузкой (лимит 384 МБ) и два потока (`RAYON_NUM_THREADS`); на одноядерном VPS — `RAYON_NUM_THREADS=1`, профиль считается за ~100–200 мс, очередь всё равно в фоне. `EMBEDDINGS=off` в `.env` выключает эмбеддинги целиком (сервис тогда можно не поднимать: `--scale embedder=0`). Без доступа к Hugging Face — `make model` на машине с сетью, сборка с `EMBEDDER_FETCH=0` и каталог `models/` смонтирован в сервис (README, «Эмбеддер без сети»).
- `KC_HOSTNAME` (публичный адрес Keycloak), `SITE_URL` (адрес сайта для canonical-ссылок, sitemap, OG-картинок и llms.txt) и `GRAFANA_ADMIN_USER` / `GRAFANA_ADMIN_PASSWORD` подставляются в `deploy/compose.yml` из окружения shell или из `deploy/.env`, а не из корневого `.env`. `SITE_URL` попадает в образ web при сборке: после смены — пересобрать web.
- В `deploy/Caddyfile` — домен вместо `:8080` и без `auto_https off`, в `deploy/compose.yml` у `caddy` — порты 80 и 443: Caddy сам получит TLS-сертификат.
- Почта: `SMTP_URL=smtps://логин:пароль@сервер:465` (или `smtp://…:587?tls=required`) и `MAIL_FROM` — адрес этого ящика либо домена с записями SPF и DKIM; без этого письма остаются в Mailpit.
- Готовый вариант всего этого — оверлей демо-сервера (`deploy/compose.demo.yml`, `deploy/Caddyfile.demo`, `deploy/render_env.py`, README «Демо-сервер»): один домен и TLS от Let's Encrypt, ФСП ID (Keycloak) по пути `/auth` того же домена, демо-почта под паролем по `/mail/`. Порты остальных сервисов привязаны к 127.0.0.1: Docker публикует порты в обход ufw, и правило файрвола их бы не закрыло.

12.4. Производительность и кеширование

Пул кандидатов и карточки. Подборка, поиск по банку, рекомендованные категории и объяснение в профиле ранжируют всех кандидатов специализации. Раньше пул собирался из базы на каждый запрос; теперь каждая специализация хранится в памяти API компактным снимком, а запрос только фильтрует его и считает баллы. В том же снимке лежат карточки показанных кандидатов: при каждом показе из базы читаются только живые поля — опубликован ли профиль, статус поиска, флаги приватности, последняя активность, возраст и отношение компании, — а навыки, ФСП, опыт, задания и бейдж прокторинга берутся из памяти. Снимок вместе с карточками сбрасывается после любой записи, меняющей выдачу или карточку: профиль, навыки, категория, приватность, согласия, ФСП, сдача и оценка задания, загрузка видео после конца теста, удаление аккаунта. Между копиями сервера изменение доходит через счётчик в Valkey не дольше чем за 250 мс, страховочный срок жизни снимка — 30 с; пока один запрос перезагружает снимок, остальных до 5 с обслуживает прежний (того же поколения — записей с тех пор не было). Отзыв согласия, скрытие профиля, имени, данных ФСП или компаний в опыте действуют с первого же запроса на любой копии: живые поля сверяются при каждом показе, а карточка, собранная при других флагах приватности, читается заново. Подробно — docs/architecture.md §12.2.

HTTP. Справочники — `public, max-age=300, stale-while-revalidate=86400` с ETag и ответом 304, опубликованные вакансии — `max-age=30`, всё личное — `private, no-store`. Caddy сжимает страницы и статику уровнем `zstd better`, а JSON API — уровнем zstd по умолчанию (вдвое дешевле по процессору при 1–11 % лишних байтов), отдаёт файлы `public/` с суточным кешем, а предрендеренным страницам вместо годового `s-maxage` ставит проверку ETag в браузере и минуту в общем кеше — чтобы после выкладки CDN не раздавал HTML со ссылками на старые чанки (docs/architecture.md §12.1).

Как мерили. Нативная release-сборка API на ноутбуке Apple M5 (4 производительных и 6 энергоэффективных ядер), PostgreSQL 14 и Valkey на той же машине; база API, Postgres и генератор нагрузки делят эти 10 ядер. Нагрузка — wrk, 50 соединений, 2 с прогрева и 10 с замера. «До» — код до этой правки, «после» — с ней; стороны

запускаются поочерёдно, каждая на своей копии одной и той же базы (`CREATE DATABASE … TEMPLATE`). На 240 кандидатах — медиана трёх прогонов, на 10 000 — один прогон. Генератор нагрузки не узкое место: на запросах к базе oha с 2 и 10 потоками и wrk с 2 и 4 потоками дают одно и то же; на самых дешёвых маршрутах четыре потока wrk отнимают производительные ядра у сервера, поэтому потолок ниже снят двумя. Между прогонами одного и того же кода разброс на этом стенде — до 20–30 %. Ответы до и после совпадают: 116 сравнений на каждом наборе данных — подборки с фильтрами, поиск, рекомендованные категории, профили с объяснением и полные тела сохранённой подборки, профиля и страницы поиска (числа до 0,001).

Потолок. Без базы — `/healthz` и справочник `/catalog` из памяти процесса — было ~80 000 и ~68 000 запросов в секунду, стало ~150 000 и ~140 000. Маршрут с авторизацией без запросов к базе (все категории работодателя: подпись JWT, проверка отзыва токена в Valkey, кеш) — ~41 000 → ~100 000. `/me` с двумя запросами к базе — ~12 500 → ~21 000. Процессорное время API на `/healthz` — 105 → 28 мкс на запрос. Через Docker и Caddy потолок задают они, а не API: с хоста через проброс порта Colima `/healthz` — ~17 000 запросов в секунду, изнутри виртуальной машины через Caddy — 22 000 (без JSON-журнала каждого запроса — 38 000), прямо в API — 71 000.

Почему тяжёлые запросы — тысячи, а не десятки тысяч. Страница поиска — 20 карточек кандидатов, сохранённая подборка — 30 карточек и 30 объяснений (~125 КБ JSON). До кеша карточек каждая страница собирала навыки, задания, бейдж прокторинга, отношение компании и видимость в базе — 2,2–2,5 мс процессора Postgres на страницу поиска под нагрузкой, и поиск упирался в базу. С кешем на страницу остаётся один лёгкий запрос живых полей — 0,4–0,55 мс процессора Postgres; остальное — работа API: копия карточек из памяти, ранжирование, JSON (0,4–0,55 мс процессора API мс). От размера банка это почти не зависит — ранжирование 10 000 кандидатов в памяти занимает доли миллисекунды. Пересборку одной подборки 50 клиентов делают по очереди: пересборки одной потребности идут под блокировкой её строки.

Что показал профиль (samply, on-CPU флеймграфы под нагрузкой): - `/healthz`: две трети процессора — клонирование и удаление стека middleware маршрута и `AppState` из 12 `Arc`. axum клонирует стек маршрута на каждый запрос и ещё раз внутри каждого `from_fn`, а счётчики ссылок общие для всех ядер. - Каждый запрос к базе — два обмена: sqlx по умолчанию пингует соединение перед выдачей из пула. - Карточки — четыре запроса (карточки, отношение компании, бейджи прокторинга, живая видимость) и ещё один — компания работодателя; основной группировал широкие строки кандидатов по навыкам. - Профиль одного кандидата: Postgres заново планировал запрос карточек на каждый вызов и не переходил на общий план — 0,5 мс вместо 0,12 мс. - Проверка подписи JWT (Ed25519, ~50 мкс) — четверть процессора на маршрутах без базы. - Показ подборки: объяснения из `jsonb` переводились в текст, разбирались в `serde_json::Value`, потом в структуру и сериализовались снова — 30–40 % процессора API на этом маршруте. - Пересборка: 30 вставок по одной под блокировкой строки потребности. - Не узкие места: журнал (построчного журнала запросов нет; `info,api=debug` в JSON и `warn` дают одно и то же), метрики (~1,5 % на `/healthz`), Valkey (одна проверка отзыва токена на запрос), ранжирование и сортировка (до 12 % процессора API на поиске).

Что сделано (docs/architecture.md §12.3): `AppState` — один `Arc`; общие middleware обёрнуты вокруг роутера один раз, а не вокруг каждого маршрута; `x-request-id` — без системного вызова. Пингуется только соединение, простоявшее больше секунды. Карточки — один запрос с подзапросами `LATERAL` (вдвое дешевле прежнего основного), видимость — в нём же; массив id передаётся так, чтобы Postgres закешировал общий план. Компания работодателя и проверенные JWT кешируются в памяти процесса без мьютексов; срок действия токена проверяется каждый раз, отзыв — как раньше, в Valkey. Объяснение в подборке хранится как `json` (миграция 0010) и отдаётся без разбора. Пересборка пишет 30 строк одним `INSERT … SELECT FROM UNNEST` и держит блокировку на три запроса вместо 32. Профиль строит объяснение по одному кандидату, не копируя пул.

Правка пути запроса — запросы в секунду и p95, мс:

Запрос	240 кандидатов: до	после	10 000: до	после
`/healthz` (без базы)	74 478 / 1,0	113 238 / 0,5	73 070 / 1,0	114 046 / 0,5
Справочник `/catalog` (из памяти)	67 501 / 1,1	110 813 / 0,5	64 309 / 1,2	103 319 / 0,6
`/me` (два запроса к базе)	11 796 / 4,6	17 059 / 3,2	12 582 / 4,2	17 001 / 3,2
Все категории работодателя	42 223 / 1,6	75 764 / 0,9	41 393 / 1,7	79 662 / 0,8
Публичный список вакансий	18 945 / 2,8	24 239 / 2,3	20 046 / 2,7	24 146 / 2,3
Рекомендованные категории потребности	11 550 / 4,7	24 253 / 2,9	10 348 / 6,2	20 626 / 2,7
Поиск по банку	2 036 / 40	2 725 / 47	2 666 / 24	3 847 / 41
Поиск по банку под потребность	2 235 / 33	2 755 / 44	2 016 / 898	2 903 / 643
Профиль с объяснением	3 157 / 21	7 704 / 8	3 032 / 25	9 746 / 5
Показ сохранённой подборки	1 777 / 36	2 508 / 54	1 194 / 52	2 124 / 44
Пересборка подборки (одна потребность)	313 / 238	503 / 149	142 / 499	272 / 282

Первые шесть строк сняты с четырьмя потоками wrk (см. выше про потолок). У поиска и подборки после правки выше и пропускная способность, и p95: запросы короче, и Postgres держит в работе все 16 соединений пула на 10 ядрах, общих с API и wrk. Хвост задаёт размер пула (`DATABASE_POOL_SIZE`): на 10 000 кандидатах при 32 соединениях пропускная способность поиска та же, что при 16, а p95 — 120 мс вместо 25; при 8 — на 9 % меньше запросов, зато p95 10 мс. Меньше потоков tokio, чем ядер, только хуже (4 потока: −12 % на поиске, −20 % на маршрутах без базы). Длинный хвост поиска под потребность на 10 000 в этой таблице — раз в 30 с снимок пересобирался, все 50 запросов ждали загрузку, а потом каждый заново ранжировал текст потребности. Это исправлено следующей правкой (ниже).

Docker и Caddy (240 кандидатов, wrk с хоста на :8080, один прогон «после»): поиск по банку 1 249 → 1 897 запросов в секунду, под потребность 1 224 → 1 468, профиль с объяснением 2 358 → 4 346, сохранённая подборка 830 → 1 210, пересборка 198 → 263, категории потребности 6 838 → 10 080, все категории 6 484 → 12 771, `/me` 7 777 → 9 094; `/healthz` — ~17 000 и там и там (потолок Colima). wrk не просит сжатия; браузер просит, и тогда сжатие JSON стоит Caddy заметных денег. На 8 соединениях `zstd better` тратил на ответ 0,43–0,63 мс процессора Caddy, уровень по умолчанию — 0,17–0,36 мс при ответе на 1–11 % больше (поиск 5 391 → 5 743 байта, подборка 9 237 → 10 282, вакансии 3 119 → 3 140); вакансий при этом 4 593 → 6 776 запросов в секунду. Поэтому `/api/*` сжимается уровнем по умолчанию, страницы и статика — `better`. Заодно нашлось, что при 50 одновременных ответах со сжатием Caddy с лимитом 64 МБ убивало OOM по кругу (пик — ~150 МБ): лимит поднят до 256 МБ, `GOMEMLIMIT=200MiB`.

Кеш карточек (docs/architecture.md §12.2). После правки пути запроса поиск и подборка упирались в Postgres на сборке карточек. Теперь карточка лежит в снимке пула и живёт столько же, сколько он; при показе из базы читаются только живые поля. Записи, которые меняли карточку без сброса поколения, теперь его сбрасывают (сдача и оценка задания, загрузка видео и событий целостности после конца теста — один-два сброса на тест или задание) или читаются живыми (последняя активность, возраст). Заодно: - `task_assignments` не имела индекса по кандидату — счётчик заданий в карточке проходил всю таблицу на каждого кандидата; «откликался в компанию» планировщик превращал в хешированный проход по всем откликам компании. Миграция `0012_candidate_side_indexes` и скалярный подзапрос с `LIMIT 1`. - Хвост поиска под потребность: раз в 30 с все запросы ждали перезагрузку снимка, а потом каждый заново считал `ts_rank_cd` текста потребности (на 10 000 кандидатах — десятки одинаковых запросов по 25+ мс; у старого кода это иногда перерастало в очередь на пул соединений на десятки секунд). Теперь снимок, у которого истёк только срок жизни, обслуживает запросы, пока один его перезагружает, а ранжирование текста считается одним запросом на слот.

Данные — те же снимки сида на 240 и 10 002 кандидата, к которым одинаково для всех сторон добавлено то, чего в сиде нет: приглашения и отклики демо-компании (61 и 69 на 240, 2 647 и 2 942 на 10 000), задания с решениями

и оценками (104 и 4 414), скрытые данные ФСП (у каждого 7-го), скрытые компании в опыте (у каждого 11-го), переключённый показ имени (у каждого 13-го). Поэтому «до» здесь медленнее, чем «после» прошлой таблицы. Медиана трёх чередующихся прогонов на обоих наборах, wrk с двумя потоками, 50 соединений; три стороны: код до правки, он же на копии базы с индексами миграции 0012 (созданы вручную) и новый код. `/me` карточек не касается и показывает разброс стенда. Запросы в секунду и p95, мс:

Запрос	240: до	до + индексы	после	10 000: до	до + индексы	после
Поиск по банку	2 441 / 51	2 300 / 47	7 936 / 11	651 / 97	2 500 / 64	6 500 / 28
Поиск по банку под потребность	2 639 / 53	2 784 / 43	7 223 / 9,6	489 / 687	2 218 / 385	4 660 / 23
Профиль с объяснением	7 970 / 9,2	8 625 / 6,8	9 847 / 6,1	2 709 / 93	4 007 / 40	7 807 / 9,7
Показ сохранённой подборки	2 441 / 54	2 610 / 51	5 072 / 16	331 / 280	1 616 / 71	3 483 / 32
Пересборка подборки (одна потребность)	496 / 153	514 / 148	540 / 161	336 / 213	299 / 257	252 / 361
`/me` (контроль, без карточек)	16 096 / 4,2	16 878 / 3,5	16 352 / 3,5	14 815 / 5,5	13 627 / 5,8	14 105 / 4,5

Индексы 0012 сами по себе ускоряют карточки на 10 000 кандидатах с заданиями (поиск — 651 → 2 500 запросов в секунду, показ подборки — 331 → 1 616: без индекса счётчик заданий проходил всю таблицу на каждую карточку), кеш карточек — в 2,6–3,4 раза поиск, в 2,1–2,6 — поиск под потребность, в 1,9–2,2 — показ подборки; хвост поиска под потребность на 10 000 — с 687 до 23 мс. Пересборка подборки — в пределах разброса: её держит блокировка строки потребности, и от прогона к прогону её пропускная способность скачет у всех трёх сторон. Во время части прогонов на 10 000 на той же машине шла посторонняя сборка Rust; разброс стенда виден по `/me`. Память API под этой нагрузкой — до ~80 МБ RSS на 10 000 кандидатах против ~20 МБ без кеша карточек (лимит контейнера — 384 МБ). Ответы совпадают со старым кодом: по 116 сравнений на каждом наборе (подборки с фильтрами, поиск, рекомендованные категории, профили с объяснением, полные тела подборки, профиля и страницы поиска, числа до 0,001). Отдельная проверка с прогретым кешем: после записей в базу в обход API (поколение не сбрасывается — худший случай отстающей реплики) обе версии одинаково скрывают профиль без согласия на публикацию, скрытый и закрытый для предложений профиль, подростка без согласия представителя, имя, данные ФСП и компании в опыте, показывают свежую активность — 17 и 16 проверок, все прошли, при этом карточки отдавались из кеша без перезагрузки снимка.

Что ещё можно. - Пересборка, которую 50 клиентов одновременно запускают для одной потребности, в отдельных прогонах проседает до единиц запросов в секунду — у старого и нового кода одинаково. Вероятная причина — клиенты, оборванные посреди транзакции, которая держит строку потребности (wrk рвёт соединения в конце прогона). Лечится выносом транзакции пересборки в отдельную задачу, которую обрыв клиента не прерывает, и `lock_timeout`. - Сразу после перезагрузки снимка одновременные запросы читают одни и те же карточки полным запросом; одна загрузка на страницу сократила бы этот всплеск, как у пула и ранжирования текста. - Окно блокировки пересборки — три обмена с базой. Конвейер запросов внутри транзакции сократил бы его до одного, но sqlx его не умеет; для разных потребностей пересборки и так идут параллельно. - JSON-журнал каждого запроса в Caddy съедает ~40 % его пропускной способности (22 000 → 38 000 `/healthz`); можно не писать успешные чтения `/api/*` или писать выборочно — вопрос того, что нужно эксплуатации. - Проброс порта Colima ограничивает стенд ~17 000 запросами в секунду; на Linux-сервере этого слоя нет.

Эмбеддинги в подборе (§5.6). Подбор под потребность теперь читает вектор потребности и сравнивает его с векторами профилей. Векторы профилей лежат в снимке пула, сжатые до байта на число (388 байт на профиль, 3,9 МБ на 10 000), близость всех кандидатов снимка к потребности считается один раз и кешируется, как ранги `ts_rank_cd`. Первая версия читала вектор потребности из базы на каждом запросе — и поиск под потребность потерял 13 % пропускной способности (8 212 → 7 178 запросов в секунду), профиль с объяснением — 21 % (12 364 → 9 774). Теперь вектор потребности лежит в памяти процесса до смены поколения (его сбрасывает воркер, когда пишет новый вектор), и разница ушла в разброс стенда.

Замер — как выше, но на PostgreSQL 17 с pgvector: три стороны на одинаковых копиях базы с 10 002 кандидатами — код до этой правки, новый код с `EMBEDDINGS=off` и новый код с эмбеддингами (векторы всех 10 002 профилей, 16 потребностей и 12 вакансий посчитаны: очередь на 10 030 текстов сервис эмбеддингов на четырёх потоках разобрал за 7 минут). Медиана трёх чередующихся прогонов, wrk с двумя потоками, 50 соединений. Запросы в секунду и p95, мс:

Запрос	До	Эмбеддинги выключены	Эмбеддинги включены
Поиск по банку под потребность	7 794 / 8,1	7 819 / 7,8	7 668 / 7,8
Профиль с объяснением под потребность	11 901 / 4,6	11 944 / 4,5	11 464 / 4,9
Показ сохранённой подборки	5 451 / 10,3	5 483 / 10,3	5 506 / 10,1
Пересборка подборки (одна потребность)	445 / 174	432 / 179	453 / 170
Поиск по банку без потребности	12 036 / 5,3	11 890 / 5,6	11 673 / 5,6
`/me` (контроль, без подбора)	19 551 / 2,7	19 336 / 2,8	19 197 / 2,9

Процессорное время на запрос поиска под потребность — 0,75 мс API и 0,37–0,39 мс PostgreSQL у всех трёх сторон. Разброс между сторонами — в пределах того, что показывает контрольный `/me`.

Память под нагрузкой в Docker (Linux, лимиты compose): 30 секунд поиска под потребность через Caddy с 50 соединений (4 100–4 250 запросов в секунду) и одновременно три прогона `POST /eval/pairs`, которые заставляют эмбеддер считать 65 текстов резюме и вакансий. API — 49 МиБ RSS (сразу после старта — 29 МиБ; сид демоданных на пустой базе при первом старте поднимает пик до ~250 МиБ); эмбеддер — 232 МиБ RSS, пик 255 МиБ (своя память — 178 МиБ, остальное — страницы файла весов), при лимите 384 МиБ; PostgreSQL — 86–126 МиБ своей памяти плюс файловый кеш. Всё ядро стека вместе с эмбеддером под этой нагрузкой занимает ~0,9 ГБ при сумме лимитов 1 984 МиБ: VPS на 2 ГБ хватает впритык, на 4 ГБ — с запасом, вместе с Keycloak и наблюдаемостью. Эмбеддер на двух потоках считает профиль за 50–90 мс; запросы его не ждут.

Индексы (миграции `0009_perf_indexes` и `0012_candidate_side_indexes`) — только те, что оправдал `EXPLAIN (ANALYZE, BUFFERS)` на 10 000 кандидатах, 300 компаниях, 45 000 приглашениях и 90 000 просмотров профиля. Среднее время запроса (pgbench, 50 повторов):

Запрос	Было	Стало	Что изменилось
Потребность со счётчиком приглашений (каждый запрос с потребностью)	1,53 мс	0,16 мс	`invitations (need_id)`: вместо полного прохода по приглашениям
Список потребностей компании	3,27 мс	0,16 мс	тот же индекс
Ленивое истечение приглашений (перед каждым чтением приглашений)	1,09 мс	0,23 мс	`invitations (expires_at)` по живым приглашениям
Вилка категории (проверка зарплаты, публикация вакансии)	3,67 мс	0,87 мс	`user_consents (document_id, user_id)` по действующим согласиям
Спрос в категории кандидата	14,9 мс	13,6 мс	`profile_views (candidate_id)`; остальное — кеш
Загрузка пула специализации	59 мс	16 мс	навыки собираются по кандидату, а не группировкой всего соединения (24 мс), плюс индекс по согласиям
30 карточек кандидатов при 4 400 заданиях (миграция 0012)	7,2 мс	1,7 мс	`task_assignments (candidate_id)`: счётчик заданий без прохода по всей таблице
Живые поля 30 карточек (запрос карточки из кеша)	0,72 мс	0,33 мс	«откликался в компанию» — скалярный подзапрос по `applications (candidate_id)` вместо хешированного прохода по 2 400 откликам компании

Ранжирование текста потребности по «о себе» (`ts_rank_cd`, 25 мс на специализацию) индексом не ускоряется — оно кешируется вместе со снимком, по тексту потребности. Индекс по дате создания приглашений сокращал спрос в категории ещё на 4 мс, но этот запрос теперь кешируется, и индекс не добавлен.